

Konzeption und Implementierung eines GitHub-basierten Branching-, Review- und Projektmanagementprozesses mit KI-gestützter Code-Compliance

T4_2000

Studiengang Elektrotechnik

Duale Hochschule Baden-Württemberg Ravensburg
Campus Friedrichshafen

Florian Germann

Eingereicht am: 24.08.2026

Matrikelnummer, Kurs: 7582635, TEA24

Unternehmen: fpt Systems GmbH, 88279, Amtzell

Betreuer im Unternehmen: Simon Würzer

Inhalt

Erklärung zum Einsatz von KI-gestützten Werkzeugen	1
1 Einleitung	3
2 Forschungsfragen und Methodik	6
2.1 Forschungsfragen	6
2.2 Methodisches Vorgehen	7
2.2.1 Untersuchungsdesign	7
2.2.2 Datengrundlage und Auswertungseinheit	7
2.2.3 Bewertungsmetriken	7
3 Ausgangslage und Zielbild	9
3.1 Problematik	9
3.1.1 Branching	9
3.1.2 Management	10
3.1.3 Code-Review	10
3.1.4 Entwicklungsumgebung	11
3.2 Lösungsansatz	11
3.2.1 Branching	11
3.2.2 Management	11
3.2.3 Code-Review	12
3.2.4 Entwicklungsumgebung	12
3.3 Teilfazit Ausgangslage und Zielbild	12
4 Grundlagen und Ausgangslage	14
4.1 Industriestandards für Versionskontrolle	14
4.1.1 GitHub-Flow	14
4.1.2 Git-Flow: Strukturiertes Branching für komplexe Releases	16
4.2 Richtlinien E-SW	20
4.3 Zwischenfazit zur Branching-Wahl	22
5 Evaluation des Code-Review-Ansatzes	23
5.1 Rahmenbedingungen und Testumgebung	23

5.1.1	Testinfrastruktur	23
5.1.2	Testmethodologie	24
5.2	Evaluationsergebnisse und Verbesserungszyklen	25
5.2.1	Iteration 1: Präzision der Conventions	25
5.2.2	Iteration 2: Kommentarunterdrückung bei Umbenennungen	26
5.2.3	Iteration 3: Balance zwischen Präzision und Agilität	31
5.2.4	Iteration 4: Bestandscode-Analyse	31
5.2.5	Zusammenfassende Bewertung der Iterationen	32
5.3	Zwischenfazit zur Evaluation	33
6	Entwicklungsumgebung	34
6.1	Dateiverwaltung	34
6.2	SPS-Programmierung	35
6.3	GitHub Verwaltung	36
7	Pilotierung im Projekt	38
7.1	Umsetzung im Pilotprojekt	39
7.1.1	Branching Strategien	39
7.1.2	Copilot Code-Review	40
8	Projektmanagement mit ZenHub	43
8.1	E-SW Prozess als Referenz	44
8.2	Übertragung auf die O-SW	44
8.3	Zwischenfazit zum Projektmanagement	45
9	Schulung und Wissenssicherung	46
9.1	Bedarfsermittlung	46
9.2	Schulungsumgebung: Evaluierte Optionen	47
9.3	Umsetzung der Dokumentation	48
9.3.1	Git und GitHub Grundlagen	48
9.3.2	Branching Strategien	48
9.3.3	Entwicklungsumgebung	49
9.4	Zwischenfazit zur Schulung und Wissenssicherung	50
10	Einführung im Team	51
11	Diskussion, Limitationen und Wirtschaftlichkeit	52

11.1 Einordnung der Ergebnisse	52
11.2 Limitationen und Validität	52
11.3 Wirtschaftliche Einordnung	53
11.4 Governance-Empfehlungen für den Dauerbetrieb	55
12 Fazit	56
A Literatur	58
B Glossar	59



Erklärung zum Einsatz von KI-gestützten Werkzeugen

Im Rahmen dieser Projektarbeit wurden KI-gestützte Werkzeuge ausschließlich als unterstützende Hilfsmittel eingesetzt. Die folgende Tabelle gibt einen Überblick über die genutzten Systeme und deren konkreten Verwendungszweck:

Werkzeug	Anbieter	Verwendungszweck
GitHub Copilot	Microsoft / GitHub	Automatisiertes Code-Review anhand der Coding-Conventions bei Pull Requests
GitHub Copilot Chat	Microsoft / GitHub	Analyse von Review-Problemen, Recherche zu Lösungsansätzen, Überprüfung von Rechtschreibung
GitHub Copilot in Visual Studio Code	Microsoft / GitHub	Rechtschreib- und Grammatikprüfung, Ausführen lokaler Agents
GitHub Copilot Agent	Microsoft / GitHub	Tiefergehende Repository-Analyse zur Ursachenforschung bei Kommentarunterdrückung
Microsoft Copilot „Prompt Coach,,	Microsoft	Unterstützung bei der KI-kompatiblen Überarbeitung und Präzisierung der Coding-Conventions

Tabelle 1 – Eingesetzte KI-Werkzeuge

Alle durch KI-Werkzeuge generierten Ausgaben wurden vom Autor inhaltlich geprüft, bewertet und erst nach kritischer Sichtung übernommen. Eine automatische oder ungeprüfte Übernahme von KI-Inhalten erfolgte nicht. Die inhaltliche Verantwortung für diese Arbeit liegt vollständig beim Autor.

KI-Werkzeuge wurden nicht zur Erstellung des Fließtextes dieser Arbeit eingesetzt. Sie wurden unterstützend für Review-Analysen sowie für Rechtschreib- und Grammatikprüfungen verwendet.



Einleitung

Strukturierte Quellcodeverwaltung ist eine Grundvoraussetzung für moderne Softwareentwicklung in verteilten Teams. In solchen Umgebungen muss jederzeit dokumentiert sein, welche Änderungen wann und von wem durchgeführt wurden. Git-basierte Plattformen wie GitHub bieten dafür eine etablierte Lösung.

GitHub ermöglicht zentrale Ablage und gemeinsame Weiterentwicklung von Projekten. Durch die Git-Versionskontrolle lassen sich Änderungen transparent dokumentieren und frühere Zustände reproduzieren. Dies steigert in komplexen Projekten die Nachvollziehbarkeit und reduziert Abstimmungsaufwand erheblich.

Branches sind ein zentrales Konzept dieser Versionskontrolle. Sie ermöglichen es, neue Funktionen, Fehlerbehebungen oder Experimente isoliert umzusetzen. Nach Prüfung und Freigabe fließen diese Änderungen in die Hauptentwicklungslinie zurück. So arbeiten mehrere Personen parallel, ohne sich gegenseitig zu behindern.

Die Kombination aus Versionskontrolle, nachvollziehbarer Historie und parallelen Entwicklungszweigen schafft eine robuste Grundlage für Teamarbeit.

In der Praxis hängt die Effektivität von GitHub stark von den gewählten Prozessen ab. Insbesondere prägen zwei Faktoren den Erfolg: erstens eine passende Branching-Strategie und zweitens ein funktionierendes Team-Management. Diese definieren, wie Features, Bugfixes und Releases vorbereitet und integriert werden, und zwar nicht nur technisch, sondern auch organisatorisch.

Eine gute Branching-Strategie allein genügt jedoch nicht. Ergänzend erforderlich ist strukturierte Aufgabenplanung im Team. Projektmanagement-Werkzeuge unterstützen dabei durch: transparente Priorisierung von Arbeitspaketen, klare Verantwortlichkeitszuweisung und sichtbare Fortschrittsverfolgung. Die Verzahnung mit Entwicklungsprozessen, automatisierten Workflows und übersichtlichen Boards schafft gemeinsames Verständnis für anstehende Aufgaben und potenzielle Engpässe. Dies fördert Zusammenarbeit und steigert Qualität sowie Implementierungsgeschwindigkeit.

Im industriellen Umfeld der Anlagenprogrammierung sind diese Anforderungen besonders ausgeprägt. Projekte bestehen aus langlaufenden Meilensteinen, mehreren beteiligten Rollen und einer hohen Abhängigkeit zwischen Softwareständen, Inbetriebnahme und Releaseplanung. Schon kleine Unklarheiten bei Branch-Zuständigkeiten, Review-Pfaden oder Freigaben können in dieser Konstellation zu Verzögerungen führen, weil Fehler oft erst in späteren Integrationsphasen sichtbar werden. Ein methodisch konsistenter Entwicklungsprozess ist deshalb nicht nur ein organisatorischer Vorteil, sondern ein zentraler Beitrag zur technischen Risikoreduktion.

Ziel dieser Arbeit ist die Konzeption und praktische Überführung eines GitHub-basierten Prozessmodells für die O-SW. Im Fokus stehen drei zusammenhängende Handlungsfelder: erstens eine tragfähige Branching-Strategie für meilensteinorientierte Projekte, zweitens ein skalierbarer Code-Review-Ansatz mit KI-Unterstützung und drittens die organisatorische Einbettung über Team-Management und Schulung. Die Arbeit verfolgt dabei ausdrücklich einen anwendungsnahen Ansatz: Konzepte werden nicht nur theoretisch bewertet, sondern in Test- und Pilotphasen erprobt.

Die Abgrenzung liegt darin, dass keine vollständige Reorganisation aller Unternehmensprozesse vorgenommen wird. Stattdessen konzentriert sich die Untersuchung auf den Entwicklungs- und Freigabeprozess rund um GitHub, Copilot-Reviews, ZenHub-gestützte Planung sowie die Kombination aus VS Code und TwinCat XAE Shell. Themen wie allgemeine IT-Infrastruktur, unternehmensweite Governance außerhalb des Projektkontexts oder eine vollständige wirtschaftliche Kostenrechnung werden nur insoweit betrachtet, wie sie die technische Umsetzbarkeit der vorgeschlagenen Prozessbausteine direkt beeinflussen.

Zur didaktisch klaren Einordnung folgt die Arbeit der Abfolge Ausgangslage, konzeptioneller Lösungsansatz, fachliche Grundlagen, Evaluation im

Testkontext, Pilotierung im Projekt, organisatorische Implementierung und Schulung sowie abschließendes Fazit.



Forschungsfragen und Methodik

2.1 Forschungsfragen

Um die Zielerreichung nachvollziehbar zu prüfen, werden die folgenden Forschungsfragen zugrunde gelegt:

- **F1:** Inwieweit verbessert ein angepasstes Git-Flow-Modell die Nachvollziehbarkeit und Stabilität von Integrations- und Freigabeprozessen in der [O-SW](#)?
- **F2:** Welchen messbaren Beitrag leisten Copilot-gestützte Reviews zur Erkennung formaler Regelverstöße im Vergleich zur rein manuellen Prüfung?
- **F3:** Inwieweit reduziert die Einführung von ZenHub den organisatorischen Abstimmungsaufwand und erhöht die Transparenz über Verantwortlichkeiten, Prioritäten und Abhängigkeiten?
- **F4:** Unter welchen organisatorischen und technischen Randbedingungen ist das kombinierte Prozessmodell langfristig im Teamalltag tragfähig?

Die Kapitel sind entlang dieser Fragen strukturiert: F1 wird primär über Branching-Konzeption und Pilotierung beantwortet, F2 über die Evaluation des Review-Ansatzes, F3 über die Management- und ZenHub-Abschnitte sowie F4 über Einführung, Schulung, Limitationen und Wirtschaftlichkeitsbetrachtung.

2.2 Methodisches Vorgehen

2.2.1 Untersuchungsdesign

Die Arbeit folgt einem anwendungsorientierten, iterativen Design mit den Phasen Analyse, Konzeption, Evaluation im Testkontext, Pilotierung im Projekt und organisatorischer Verstetigung. Methodisch handelt es sich um eine praxisnahe Fallstudie im industriellen Umfeld mit formativer Verbesserung über mehrere Iterationen.

Die Evaluation kombiniert qualitative und semiquantitative Elemente. Qualitativ werden Beobachtungen aus Reviews, Pilotbetrieb und Teamrückmeldungen ausgewertet. Semiquantitativ werden wiederkehrende Befundtypen, Review-Verhalten und Prozessindikatoren systematisch kategorisiert und zwischen Iterationen verglichen.

2.2.2 Datengrundlage und Auswertungseinheit

Die Datengrundlage umfasst:

- Pull-Request-Reviews und zugehörige Befunde aus dem Test-Repository.
- Beobachtungen und Rückmeldungen aus dem Pilotprojekt der [O-SW](#).
- Prozessartefakte aus Branching-, Review- und Planungsabläufen (z. B. Branch-Strukturen, Boards, Merge-Verläufe).

Die Auswertungseinheit ist jeweils ein klar abgegrenzter Iterationszyklus beziehungsweise ein definierter Projektabschnitt. Dadurch bleiben Ursache-Wirkungs-Bezüge zwischen Maßnahme und beobachtetem Ergebnis nachvollziehbar.

2.2.3 Bewertungsmetriken

Zur Vergleichbarkeit werden die folgenden Metriken verwendet:

Metrik	Bedeutung im Kontext dieser Arbeit
Review-Abdeckung	Anteil der relevanten Änderungen, die im PR-Prozess durch Review-Regeln adressiert werden.
Befundpräzision	Grad der fachlich korrekten Zuordnung von Hinweisen zu konkreten Conventions.
Unterdrückungsanfälligkeit	Häufigkeit, mit der formal korrekte Hinweise aufgrund von Diff-Eigenschaften nicht sichtbar werden.
Prozessdurchlauf	Tendenz der Bearbeitungsdauer von Änderung bis Merge über standardisierte PR-Schritte.
Koordinationsaufwand	Erforderliche Abstimmungsintensität zwischen Rollen bei Priorisierung, Review und Freigabe.

Tabelle 2 – Bewertungsmetriken für Evaluation und Pilotierung

Die Metriken werden überwiegend als Trends (niedrig, mittel, hoch; verbessert, unverändert, verschlechtert) bewertet, sofern keine belastbaren numerischen Reihen vorliegen.



Ausgangslage und Zielbild

3.1 Problematik

3.1.1 Branching

In der Abteilung **O-SW** existiert bislang keine verbindliche und einheitliche Vorgehensweise für GitHub-Branches. Obwohl die **E-SW** Richtlinien bereits bei internen Bibliotheken und Template-Projekten eingesetzt werden, zeigt sich in der Praxis: Diese Vorgaben passen nicht auf die Anlagenentwicklung. Die Entwicklungsabläufe unterscheiden sich strukturell und inhaltlich, was zu alltagsprägenden Unklarheiten führt.

Ohne diese Passgenauigkeit treffen Entwickler Branching-Entscheidungen zunehmend individuell. Das führt zu uneinheitlichen Abläufen, erschwert die Nachvollziehbarkeit von Änderungen und erhöht das Konfliktrisiko beim Zusammenführen von Code. Gleichzeitig sinkt die Zusammenarbeitseffizienz, da gemeinsame Regeln keine durchgehende Anwendung finden.

Im Tagesgeschäft zeigt sich dies insbesondere bei der Frage, wann Änderungen direkt in develop integriert werden dürfen und wann zunächst in enger abgegrenzten Feature- oder Bugfix-Branches gearbeitet werden muss. Fehlen dafür verbindliche Kriterien, entstehen in einzelnen Projekten unterschiedliche Interpretationen. Dadurch wächst der Abstimmungsbedarf zwischen den Beteiligten, Pull Requests werden uneinheitlich vorbereitet und Freigabeent-

scheidungen werden stärker personenabhängig. Die Folge ist kein einzelner großer Fehler, sondern eine Vielzahl kleiner Reibungsverluste, die den Entwicklungsfluss über die Projektlaufzeit hinweg messbar verlangsamen.

3.1.2 Management

Im Bereich Management nutzt die **O-SW** derzeit keine dedizierte Team-Management-Software. Die Release-Planung erfolgt vornehmlich über mündliche Absprachen und E-Mail-Verteiler. Dies ist in der Praxis fehleranfällig und nur begrenzt transparent. Zusätzlich entstehen Engpässe, wenn neue **FlexOp**-Versionen verbreitet werden und notwendige Vorbereitung nicht rechtzeitig anläuft.

Insbesondere bei kurzfristigen Prioritätsänderungen fehlt ohne zentrales Board häufig die direkte Sicht auf Abhängigkeiten zwischen Aufgaben. Informationen liegen verteilt in Postfächern, Gesprächsnotizen oder Einzelsystemen vor und müssen manuell zusammengeführt werden. Das erhöht nicht nur den Koordinationsaufwand für Teamleitung und Entwickler, sondern erschwert auch eine belastbare Fortschrittsbewertung. Für einen stabilen Releaseprozess ist jedoch entscheidend, dass Verantwortlichkeiten, Status und nächste Schritte jederzeit eindeutig nachvollziehbar sind.

3.1.3 Code-Review

Bei der Codequalität zeigt sich trotz formulierter Coding-Conventions ein wiederkehrendes Problem: Falsch benannte Variablen und unzureichend dokumentierte Code-Passagen gelangen in Projekte. Die Ursache liegt darin, dass die Regelkontrolle bisher nur begrenzt und überwiegend manuell stattfindet. Für **TwinCat** fehlen klassische Linter-Werkzeuge oder vergleichbare Lösungen, so dass automatisierte Qualitätssicherung deutlich erschwert ist.

Die manuelle Kontrolle ist fachlich notwendig, skaliert aber mit zunehmender Projektlast nur eingeschränkt. Reviewende müssen gleichzeitig auf funktionale Korrektheit, Lesbarkeit, Namenskonventionen und Dokumentationsqualität achten. Ohne unterstützende Automatisierung steigt damit das Risiko, dass formale Regelverstöße erst spät entdeckt werden oder im Zeitdruck unbeabsichtigt übersehen werden. Genau an dieser Stelle besteht ein hoher Hebel für KI-gestützte Reviews, sofern deren Ergebnisse nachvollziehbar und reproduzierbar in den Pull-Request-Prozess eingebettet werden.

3.1.4 Entwicklungsumgebung

In der **O-SW** besteht das Problem fehlender Entwicklungsumgebung-Einheit. Die Ursache: Lizenzen der bisherigen Lösung sind abgelaufen; Kostengründe und lange Update-Zyklen machen Neuentwicklung erforderlich. Aktuell existiert kein etabliertes Nachfolgesystem.

Die bisherige Werkzeuglandschaft war über mehrere Anwendungen verteilt, sodass Dateiverwaltung, Versionskontrolle und Programmierung in getrennten Oberflächen stattfanden. Dieser Medienbruch erschwert konsistente Arbeitsabläufe, weil Kontextwechsel zwischen Tools zusätzliche Zeit benötigen und Fehlerquellen im Prozess erzeugen. Für eine belastbare Teamarbeit wird daher eine Umgebung benötigt, die den kollaborativen GitHub-Workflow unterstützt und gleichzeitig die technischen Anforderungen der SPS-Programmierung ohne Lizenzhürden abdeckt.

Aus diesen vier Problemfeldern ergibt sich der Bedarf nach einem gemeinsamen, technisch umsetzbaren und von der gesamten Abteilung getragenen Lösungsansatz.

3.2 Lösungsansatz

3.2.1 Branching

In der **E-SW** sind bereits funktionierende Branching-Strategien etabliert. Diese werden in der **O-SW** heute schon bei internen Bibliotheken und Template-Projekten eingesetzt und sollen im nächsten Schritt auf die Anlagenentwicklung ausgeweitet werden.

Dazu werden die bestehenden Regeln systematisch darauf geprüft, inwieweit sie zu den spezifischen Abläufen der **O-SW** passen. Ziel ist eine konsistente Richtlinie, die fachlich tragfähig ist und unternehmensweit einheitlich angewendet werden kann.

3.2.2 Management

Für die aktuell wenig strukturierten Management-Abläufe in der **O-SW** liegt innerhalb der **fpt Systems GmbH** bereits ein erprobter Lösungsweg vor: In der **E-SW** wird seit einiger Zeit die Team-Management-Software ZenHub ge-

nutzt. Damit werden Aufgaben priorisiert, Teammitgliedern zugewiesen und Releases von **FlexOp** sowie weiteren Hochsprachen-Projekten geplant.

ZenHub soll deshalb auch in der **O-SW** eingeführt werden. Ziel ist es, die Weiterentwicklung der Standardbibliotheken und die Pflege der Template-Projekte für die SPS-Programmierung besser zu strukturieren. Gleichzeitig soll die Release-Abstimmung mit der **E-SW** verlässlicher werden.

3.2.3 Code-Review

Zur Verbesserung der Code-Qualität wird ein flexibler Ansatz verfolgt, der klassische Linter ergänzt beziehungsweise in Teilbereichen ersetzt. In Abstimmung mit dem KI-Experten der **E-SW** Ralf Scheyerle und dem in der **O-SW** für Richtlinien verantwortlichen Mitarbeiter Robin Hettel wurde daher die Einführung von GitHub-Copilot-gestützten Code-Reviews vorgeschlagen. Dieser Ansatz lässt sich an projektspezifische Anforderungen anpassen und unterstützt eine effizientere Qualitätssicherung.

3.2.4 Entwicklungsumgebung

Um in der **O-SW** wieder nahtlos entwickeln zu können, wird derzeit eine neue Umgebung aufgebaut. Bisher kamen Visual Studio 2017 in Kombination mit dem Windows Explorer, GitHub Desktop oder Sourcetree sowie ein in Visual Studio integriertes Werkzeug für KUKA-Steuerungen zum Einsatz. Das firmeneigene integrierte Werkzeug FlexManager ist jedoch veraltet und kann nicht auf neuere Visual-Studio-Versionen aktualisiert werden. Da deshalb ohnehin eine Neuentwicklung erforderlich wäre und die Visual-Studio-Lizenzen auslaufen, wurde die Konzeption einer neuen Entwicklungsumgebung beschlossen. Die Entscheidung fiel auf Visual Studio Code in Kombination mit der TwinCat XAE Shell, da sich VS Code durch Extensions bedarfsgerecht erweitern lässt.

3.3 Teilfazit Ausgangslage und Zielbild

Erkenntnis: Die identifizierten Problemfelder lassen sich nur durch ein abgestimmtes Maßnahmenbündel aus Branching-Regeln, Managementprozess, Review-Methodik und geeigneter Entwicklungsumgebung adressieren.

Implikation: Damit die eingeführten Verbesserungen im Alltag zuverlässig funktionieren, müssen alle Mitarbeitenden nach der Implementierung informiert und entsprechend geschult werden. Zusätzlich ist eine zentrale, leicht zugängliche Dokumentation der neuen Abläufe notwendig, damit die Anwendung langfristig konsistent bleibt.

Übergang: Vor diesem Hintergrund werden im folgenden Kapitel die fachlichen Grundlagen und Referenzmodelle systematisch eingeordnet.

4

Grundlagen und Ausgangslage

4.1 Industriestandards für Versionskontrolle

In der Softwareentwicklung haben sich mehrere Branching-Muster etabliert. Die folgenden zwei Modelle sind am weitesten verbreitet und werden im Hinblick auf ihre Eignung für den hier vorliegenden Kontext untersucht.

4.1.1 GitHub-Flow

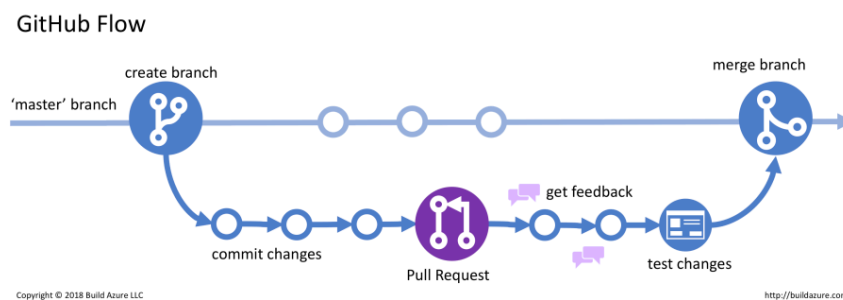


Abbildung 1 – GitHub Flow

GitHub-Flow ist ein schlankes, auf kontinuierliche Auslieferung ausgerichtetes Branching-Modell. Es eignet sich besonders für Teams, die schnell auf neue Anforderungen reagieren und Änderungen zeitnah bereitstellen müssen. Charakteristisch sind kurze Feature-Branche, häufige Pull Requests und automatisierte Tests, sodass neue Anpassungen zügig in den Main-Branch integriert werden können. Entsprechend wird GitHub-Flow vor allem in

DevOps-nahen Umgebungen und Projekten mit hoher Release-Frequenz, etwa bei Web-Anwendungen, häufig eingesetzt.

Die Ausgestaltung Pull-Request-basierter Zusammenarbeit und ihre Auswirkungen auf den Entwicklungsprozess sind in der Literatur ausführlich untersucht worden [1].

Branch	Beschreibung
main(master)	Enthält fertige Releases Stabil: Änderungen nur aus feature/* per PR
feature/*	Entwicklungsbranch für einzelne Features oder Fixes werden direkt von main erstellt nach Fertigstellung direkt PR in main

Tabelle 3 – GitHub-Flow Branches

Tabelle 3 zeigt die Branch-Typen und ihre Aufgaben; Abbildung 1 visualisiert den Ablauf. Beide verdeutlichen: GitHub-Flow ist optimiert für schnelle, häufige Feature-Releases. Ob dieses Modell geeignet ist, hängt von organisatorischen und technischen Rahmenbedingungen ab.

VORTEILE

GitHub-Flow überzeugt vor allem durch seine geringe Einstiegshürde. Da nur zwei Branch-Typen existieren, ist der Lernaufwand minimal und neue Teammitglieder können schnell produktiv mitarbeiten. Die konsequente Nutzung von Pull Requests für jede Änderung fördert kontinuierliches Feedback und ermöglicht eine zügige Integration abgeschlossener Features direkt in den Main-Branch. Teams, die auf automatisierte Tests und Continuous Deployment setzen, profitieren davon besonders: Jede Änderung durchläuft den gleichen, klar definierten Pfad von der Entwicklung bis zur Auslieferung.

NACHTEILE

Der Fokus auf einfache, häufige Releases wird zum Nachteil, sobald eine Codebasis mehrere parallele Versionsstände unterstützen muss. Da kein eigenständiger Develop-Branch existiert, landen alle Änderungen unmittelbar in main.

Bei größeren Teams oder komplexen Releases kann das zu einem instabilen Hauptzweig führen, wenn mehrere unfertige Features gleichzeitig integriert werden. Auch Hotfix-Szenarien bei bereits veröffentlichten Versionen lassen sich im Modell nicht sauber abbilden, da ein separater, produktionsnaher Branch fehlt.

BEWERTUNG FÜR DEN O-SW KONTEXT

Für die O-SW ist GitHub-Flow wenig geeignet. Die Abteilung entwickelt Anlagensteuerungen, die an klar definierten Projektmeilensteinen freigegeben werden. Eine kontinuierliche Auslieferung findet nicht statt; stattdessen müssen einzelne Programmstände über längere Zeiträume stabil gehalten und unabhängig voneinander gepflegt werden können. Genau diese Anforderung, nämlich parallele stabile Versionsstände mit strukturierter Release-Vorbereitung, kann GitHub-Flow strukturell nicht erfüllen. Die fehlende Trennung zwischen laufender Entwicklung und freigegebenen Ständen würde die Nachvollziehbarkeit in der Projektdokumentation erschweren und das Risiko erhöhen, dass nicht freigegebene Änderungen in produktive Meilensteine einfließen. Strukturiertere Modelle wie Git-Flow sind daher für diesen Einsatzbereich zu untersuchen.

4.1.2 Git-Flow: Strukturiertes Branching für komplexe Releases

Git-Flow ist ein strukturiertes Branching-Modell, das Entwicklungsabläufe klar trennt und stabile Releases absichert. Vorgestellt 2010 von Vincent Driesen ([2]), wird es in Projekten mit parallelen Features, Hotfixes und Versionen am häufigsten angewendet.

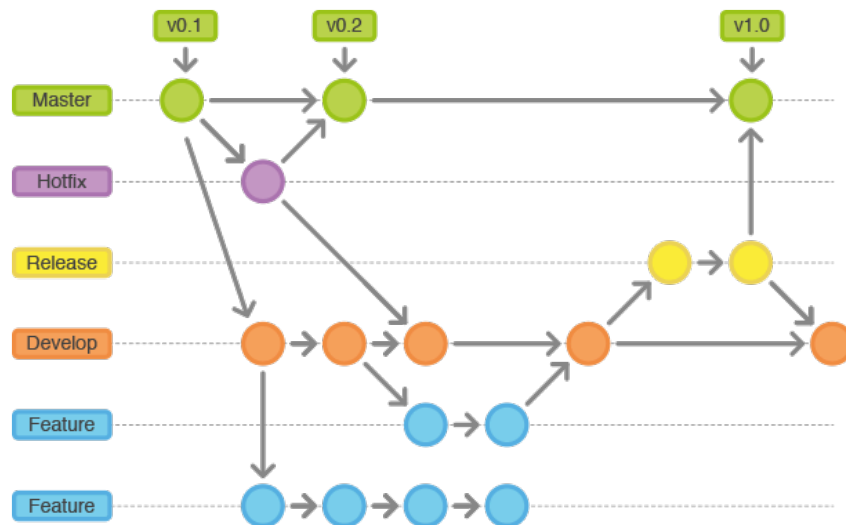


Abbildung 2 – Git Flow

Das Modell basiert auf mehreren langlebigen Branches mit definierten Rollen. Abbildung 2 zeigt deren Zusammenspiel, Tabelle 4 beschreibt die einzelnen Branches und ihr Verhalten.

Branch	Beschreibung
main(master)	Enthält fertige Releases Stabil: Änderungen nur aus release/* per PR Änderungen in main müssen zurück in develop fließen.
develop	Grundlage aller Entwicklungsbranches Stabil: Änderungen nur als PR
feature / *	Entwicklungsbranches neuer Features In diesen wird gearbeitet Strukturierung über Subbranches möglich
bugfix / *	Entwicklungsbranch für Bugfixes Wird von develop abgespalten um Bugs zu entfernen
release / *	Vorbereitung für den Release Wird von develop abgespalten Änderungen fließen wieder in develop z. B. Erstellung von Release Notes
hotfix / *	Hotfix von main Änderungen fließen auch in develop

Tabelle 4 – Git-Flow Branches

Dieses Modell unterstützt strukturierte Abläufe mit klar umrissenen, aufbauend verknüpften Releases. Für Rolling-Release-Ansätze ist es weniger geeignet, da die Verweildauer bis zur Freigabe tendenziell länger ist. Da die **O-SW** Releases primär an Projektmeilensteinen anknüpft, ist Git-Flow für diesen Einsatzfall gut geeignet.

VORTEILE

Git-Flow bietet eine klare strukturelle Trennung zwischen laufender Entwicklung, Release-Vorbereitung und hotfix-nahen Eingriffen. Durch den Develop-Branch bleibt der Integrationsstand jederzeit stabil, ohne dass fertige Features auf eine Freigabe warten müssen. Feature- und Bugfix-Banches isolieren einzelne Arbeitspakete so, dass parallele Arbeiten sich nicht gegenseitig blockieren. Der Release-Branch ermöglicht gezieltes Aushärten vor der Auslieferung, sodass finale Tests und Dokumentationsanpassungen stattfinden können, ohne den weiterlaufenden Entwicklungsfluss zu unterbrechen. Für Projekte mit mehreren aktiven Versionsständen lassen sich Hotfixes direkt auf Release-Banches aufsetzen und anschließend sauber in Develop zurückführen.

NACHTEILE

Die Stärken des Modells gehen mit einem höheren Verwaltungsaufwand einher. Die Anzahl parallel existierender Branches steigt mit Teamgröße und Projektlaufzeit schnell an. Ohne konsequente Pflege, insbesondere beim Zurückführen von Release- und Hotfix-Banches in Develop, entstehen leicht Divergenzen, die spätere Merges erschweren. Für sehr kleine Teams oder Projekte mit ausschließlich kurzen, inkrementellen Änderungen kann der zusätzliche Prozessaufwand unverhältnismäßig sein.

BEWERTUNG FÜR DEN O-SW KONTEXT

Für die **O-SW** überwiegen die Vorteile von Git-Flow klar. Die meilensteingebundene Release-Struktur der Anlagenprogrammierung entspricht exakt dem Szenario, für das das Modell konzipiert wurde: Neue Features werden über Develop integriert und erst dann in einem dedizierten Release-Branch zur Auslieferung vorbereitet. Gleichzeitig bleibt die Möglichkeit erhalten, produktive Stände per Hotfix zu korrigieren, ohne die laufende Entwicklung zu gefährden. Da in der **O-SW** bereits Erfahrung mit Git und GitHub besteht und die Richtlinien der **E-SW** als Referenz vorliegen, ist der Einführungsaufwand überschaubar. Der Mehraufwand durch Branch-Verwaltung rechtfertigt sich durch die gewonnene Transparenz und Stabilität im Freigabeprozess.

4.2 Richtlinien E-SW

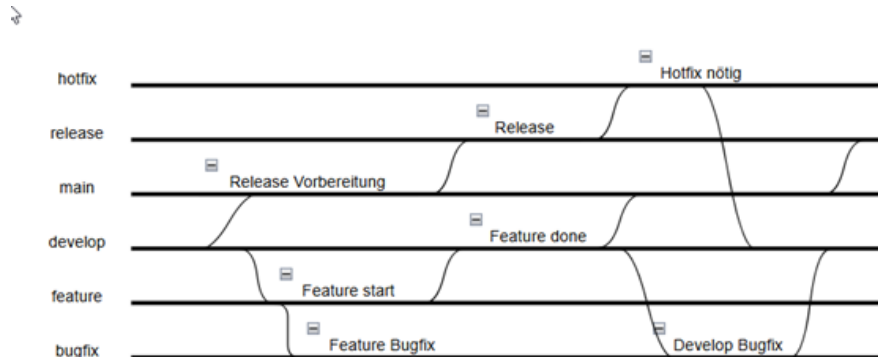


Abbildung 3 – Git Flow E-SW

Zu Beginn wurde der aktuelle Stand der Richtlinien bei Ralf Scheyerle abgefragt, der in der [E-SW](#) für dieses Thema verantwortlich ist. Da das Team nur aus fünf Personen besteht, waren die Vorgaben bislang überwiegend mündlich überliefert und lediglich in einem einfachen Word-Dokument festgehalten.

Die in Abbildung 3 dargestellte Vorgehensweise zeigt, dass die aktuelle Strategie der [E-SW](#) den Git-Flow-Prinzipien aus Abschnitt 4.1.2 weitgehend folgt. Ein wesentlicher Unterschied, der in Tabelle 5 genauer erläutert wird, liegt in der Release-Logik: Releases werden nicht in separaten `release/*`-Branches vorbereitet, sondern in `main` durchgeführt und anschließend über einen `release/*`-Branch veröffentlicht. Dadurch ist der Zugriff auf frühere Release-Stände sehr schnell möglich, ohne über Tags oder Pull Requests nach Versionsnummern suchen zu müssen. Nachteilig ist jedoch, dass sich über die Zeit viele inaktive Branches ansammeln und die Übersichtlichkeit sinkt.

Branch	Beschreibung
main	Für Releasevorbereitung Stabil: Änderungen nur aus „develop“ per PR Änderungen in main müssen zurück in develop fließen.
develop	Grundlage aller Entwicklungsbranches Stabil: Änderungen nur als PR
feature / *	Entwicklungsbranch neuer Features Verknüpfung mit ZenHub Epics über ID möglich Beispiel: feature/Fx.FlexOP-1640_SSI-Packagerstellung SubFeature (Task-Ebene): feature/Fx.FlexOP-1640_SSI-Packagerstellung/FPT.SSI-51_Externe-Steuerungsconfig
bugfix / *	Entwicklungsbranch für Bugfixes Verknüpfung mit ZenHub Bugs über ID möglich Beispiel: bugfix/Fx.FlexOP-1234
release / *	Releases von main Kann auch über Tag gelöst werden, sollte aber als Branch gezogen werden, sobald ein Hotfix nötig wird. Beispiel: release/Fx.FlexOP_v4.5.5.x
hotfix / *	Hotfix von Release-Branch Sollte wieder zurück in develop fließen Verknüpfung mit ZenHub Bug über ID Beispiel: hotfix/Fx.FlexOP_v4.5.5.12_2547

Tabelle 5 – E-SW Branches

Das Word-Dokument enthält in Abbildung 3 und Tabelle 5 die Kernelemente einer angepassten Git-Flow-Variante. Der zentrale Unterschied besteht darin, dass Releases nicht in separaten release/*-Branches vorbereitet, sondern in main erstellt und anschließend über einen release/*-Branch veröffentlicht werden. Abgesehen davon entsprechen die grundsätzlichen Vor- und Nachteile weitgehend dem klassischen Git-Flow-Modell.

4.3 Zwischenfazit zur Branching-Wahl

Auf Grundlage der Gespräche mit Robin Hettel wurde festgehalten, dass die Einführung von Git-Flow sowohl fachlich sinnvoll als auch organisatorisch gewünscht ist. Der Hauptgrund liegt darin, dass Git-Flow für wenige, größere Releases besser geeignet ist als GitHub-Flow. Zudem ist der in Abschnitt 4.2 genannte Vorteil für den vorliegenden Kontext weniger entscheidend, da Releases ausschließlich an Projektmeilensteinen stattfinden und der aktuelle, stabile Code-Stand im Vordergrund steht.



Evaluation des Code-Review-Ansatzes

GitHub Copilot kann mehr leisten als reine Codevervollständigung. Das System kann Entwickler auch dabei unterstützen, bestehende Coding-Conventions im Alltag konsequenter einzuhalten. Während der Arbeit analysiert Copilot den jeweiligen Kontext, erkennt wiederkehrende Muster und orientiert sich an projektinternen Standards. Dadurch entstehen Vorschläge, die nicht nur funktional sind, sondern auch besser zum Stil des Projekts passen. Teams profitieren dadurch von konsistenterer Codequalität, weniger manuellen Nacharbeiten und einem effizienteren Review-Prozess.

Die Bedeutung strukturierter Code-Reviews für Qualitätssicherung und Teamzusammenarbeit ist in der Forschung breit belegt [3], [4].

5.1 Rahmenbedingungen und Testumgebung

5.1.1 Testinfrastruktur

Die Evaluation findet in einem separaten Test-Repository statt, das vollständig von den produktiven Repositories isoliert ist. Diese Trennung stellt sicher, dass experimentelle Anpassungen an Conventions und Instruktionen keine Auswirkungen auf laufende Projekte haben. Innerhalb des Test-Repositories werden die Coding-Conventions als Markdown-Dateien im Verzeichnis `docs` abgelegt, die Copilot-Instruktionen befinden sich als Markdown-Dateien im Verzeichnis `.github`. Copilot greift ausschließlich auf Inhalte des jeweiligen

Repositories zu; externe Quellen oder projektübergreifende Kontexte werden nicht einbezogen.

Für die spätere produktive Nutzung ist eine zuverlässige Verteilung der Conventions und Instruktionen an alle relevanten Repositories entscheidend. Dazu wurde eine zentrale GitHub Action eingerichtet, die aktualisierte Conventions und Copilot-Instruktionen automatisch an alle Bibliotheks- und Template-Repositories der O-SW verteilt. Neue Projekte, die aus einem Template erstellt werden, erben diese Action automatisch, sodass kein manueller Einrichtungsaufwand entsteht. Der technische Mechanismus basiert auf den GitHub-Funktionen für PR-Reviews und geschützte Branches [5], [6]. Abbildung 4 illustriert den Verteilungsprozess.

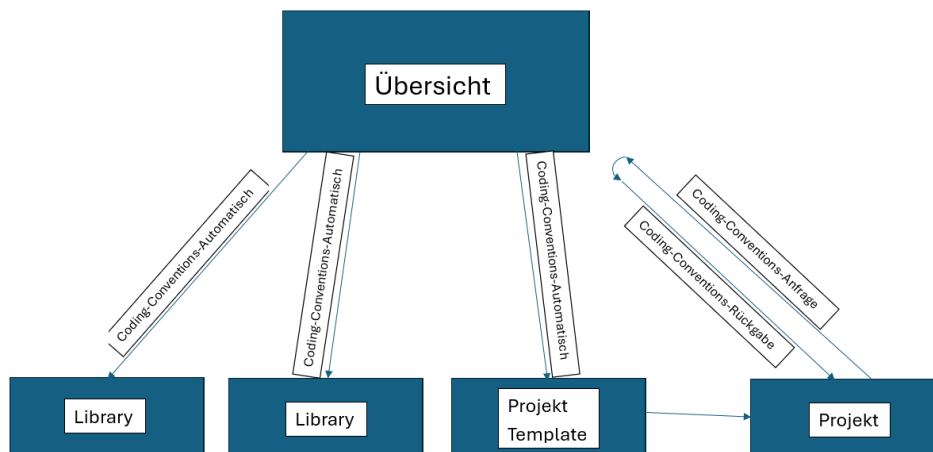


Abbildung 4 – Coding-Convention Verteilung

Zum Zeitpunkt der Evaluation sind die Actions in allen O-SW-Repositories mit Ausnahme der Projekt-Repositories vorhanden. Intensive Tests haben bestätigt, dass die automatische Übernahme beim Erstellen neuer Projekte aus Templates zuverlässig funktioniert.

5.1.2 Testmethodologie

Um aussagekräftige und vergleichbare Ergebnisse zu erzielen, wurde eine einheitliche Testmethodik festgelegt. Im Test-Repository löst jeder Pull Request auf main automatisch einen Copilot-Review aus. Für jeden Testzyklus wurden absichtlich typische Regelverstöße in den Code eingebaut, die den Conventions entsprechen, etwa falsche Variablenpräfixe, fehlende Kommentare oder abweichende Benennungsschemata. Die Bewertungsscheckliste wurde erst nach Abschluss des Reviews hinzugefügt, um zu verhindern, dass Copilot

durch den Listeninhalt beeinflusst wird und die Vergleichbarkeit zwischen den Iterationen gewahrt bleibt.

Wurde eine Iteration mit einer Anpassung der Conventions oder Instruktionen abgeschlossen, wurde der zugehörige Pull Request geschlossen und ein neuer erstellt. Dadurch startet jede Iteration unter vergleichbaren Bedingungen mit einem frischen Diff, was die Isolation der Änderungseffekte sicherstellt. Diese Vorgehensweise verhindert, dass nachträgliche Anpassungen am selben Pull Request die Bewertung der Review-Qualität verfälschen. Jede Iteration basiert damit auf einem klar abgegrenzten Testzustand mit nachvollziehbarer Historie.

Als Referenzbasis für die Iterationshistorie wurden die geschlossenen Pull Requests des Test-Repositories herangezogen. Für die vorliegende Auswertung wurden 60 aktiv getestete und anschließend geschlossene Pull Requests berücksichtigt.

Die quantitative Detailauswertung in dieser Arbeit fokussiert jedoch bewusst auf den abgegrenzten Verlauf der virtuellen Inbetriebnahme im Projekt GIRA, um eine einheitliche Vergleichsbasis innerhalb desselben Projektabschnitts sicherzustellen. Die in den Kennzahlen ausgewiesene Teilstichprobe ($n = 40$) wurde gezielt auf den einheitlich dokumentierten Projektabschnitt mit vergleichbaren Randbedingungen begrenzt.

5.2 Evaluationsergebnisse und Verbesserungszyklen

5.2.1 Iteration 1: Präzision der Conventions

Der erste Evaluationszyklus zeigte, dass Copilot grundsätzlich in der Lage ist, Coding-Conventions zu prüfen, die Gründlichkeit der Reviews jedoch weit hinter den Erwartungen zurückblieb. Eine Analyse der Befunde ergab, dass die Coding-Conventions selbst nicht präzise genug formuliert waren, um von einem KI-System zuverlässig ausgewertet zu werden. Formulierungen wie „sollte gut benannt sein,“ oder „Kommentare sind erwünscht“ lassen zu viel Interpretationsspielraum und führen zu inkonsistenten Ergebnissen.

Als Gegenmaßnahme wurden die Kapitel „TwinCat,“ und „Allgemein“ der Coding-Conventions vollständig überarbeitet. Dabei kam der Microsoft-Copilot-Agent „Prompt Coach,“ zum Einsatz, der Formulierungen auf KI-Kompatibilität prüft und Verbesserungsvorschläge liefert. Jede Regel erhielt außerdem

eine eindeutige ID nach dem Schema Kapitelnummer-Fehlernummer, beispielsweise „3.2.7-2“. Diese Referenzierbarkeit ist entscheidend, weil Copilot im Review direkt auf die jeweilige Regel verweisen und den Befund damit eindeutig begründen kann. Alle KI-generierten Formulierungsvorschläge wurden anschließend von mehreren Personen inhaltlich geprüft; eine automatische Übernahme fand nicht statt.

Nach der Überarbeitung stieg die Erkennungsqualität deutlich. Befunde ließen sich eindeutig einer Regel zuordnen, was die fachliche Bewertung beschleunigte. Der Aufwand im Review verlagerte sich damit weg von der Grundsatzfrage, ob ein Hinweis überhaupt zutrifft, hin zur produktiveren Frage, wie die Korrektur effizient umgesetzt wird.

5.2.2 Iteration 2: Kommentarunterdrückung bei Umbenennungen

Befund: Copilot erkannte Struct-Variablen-Benennungsverstöße (z. B. fehlendes `stc_`-Präfix), GitHub unterdrückte Kommentare.

Ursachenanalyse: GitHub unterdrückt Meldungen bei:

- Dateien, die als „neu hinzugefügt“ erscheinen (nach Umbenennung)
- Fehlender stabiler Diff-Verknüpfung für umbenannte Dateien
- Umbenennung als intentionale Änderung (Copilot „widerspricht“ PR-Intent)

****3.2.5-1 (Major)****
Der Datentyp JEDER Variable MUSS durch ein eindeutiges Präfix gekennzeichnet sein.

IEC-Datentyp	Präfix
-----	-----
BOOL	b_
BIT	bit_
BYTE	by_
WORD	w_
DWORD	dw_
LWORD	lw_
INT	i_
UINT	ui_
SINT	si_
USINT	usi_
DINT	di_
UDINT	udi_
LINT	li_
ULINT	uli_
REAL	r_
LREAL	lr_
TIME	t_
LTIME	lt_
DATE	d_
TIME_OF_DAY	tod_
STRING	str_
PVOID	pvo_
ARRAY	a_
STRUCT	stc_
ENUM	e_
UNION	u_
ANY	any_
REFERENCE TO	ref_
POINTER TO	ptr_

Abbildung 5 – Coding-Convention Variablen Präfixe

```
1  ### Zusatz: Referenzierung der Datentyp-Präfixe bei  
2  Benennungsverstößen (Mandatory)  
3  
4  Bei Benennungsverstößen, die Datentyp-Präfixe betreffen,  
5  MUSS als fachliche Grundlage **Tabelle 3.2.5 (Datentyp-Präfixe)**  
6  herangezogen werden.  
7  
8  Befunde MÜSSEN:  
9  
10 - das **konkret erwartete Präfix** nennen (z.B. `b_`, `i_`, `di_`)  
11 - optional explizit auf **Tabelle 3.2.5** verweisen  
12  
13 Befunde DÜRFEN NICHT:  
14 - Datentypen semantisch interpretieren (z.B. „BOOL-Präfix“)  
15 - implizite Ableitungen ohne Tabellenreferenz enthalten  
16  
17 Zulässig:  
18 - „Feldname beginnt nicht mit dem erwarteten Präfix `b_` gemäß Tabelle 3.2.5.“  
19 - „Bezeichner beginnt nicht mit dem erforderlichen Präfix `e_` (siehe Tabelle  
20 3.2.5).“
```

Listing 1 – Klausel

Nach dieser Änderung erkannte GitHub Copilot die Regelverstöße zuverlässig. Die Hinweise wurden jedoch weiterhin teilweise als „suppressed due to uncertainty“ markiert. Zur Analyse wurde die GitHub-Copilot-Funktion „Agents“ genutzt. Diese kann das gesamte Repository inklusive früherer Pull Requests als Kontext einbeziehen und ermöglicht dadurch fundiertere Auswertungen. Die Analyse zeigte, dass GitHub die Kommentare selbst unterdrückte, sobald sie auf umbenannte Dateien verwiesen. Da Git-Umbenennungen im Diff häufig als Löschen und Neu-Anlegen dargestellt werden, werden solche Befunde schneller als unsicher eingestuft, weil:

- Die gesamte Datei als „neu hinzugefügt,“ erscheint
- GitHub keine stabile Diff-Kontext-Verknüpfung für umbenannte Dateien aufbauen kann
- Das Umbenennen an sich als intentionale Änderung gilt. Copilot widerspricht dem PR-Intent

Daraus ergibt sich: Copilot erkennt den Verstoß korrekt, GitHub unterdrückt den Kommentar jedoch im Anschluss.

Die durch den Agent gelieferten Analysen wurden manuell geprüft und mit den tatsächlichen Repository-Änderungen abgeglichen, um Fehlinterpretationen auszuschließen.

Als Gegenmaßnahme wurde vorgeschlagen, die in Listing 2 gezeigte Anweisung in die GitHub-Copilot-Instruktionen aufzunehmen.

```
1 ### Pflicht-Strategie für Violations in umbenannten DUT-Dateien  
2 (Mandatory)  
3 **3.2.12-2 (Mandatory Review Behavior)**  
4  
5 Wenn ein STRUCT- oder ENUM-Typ in einer **umbenannten** DUT-Datei  
6 deklariert wird  
7 (erkennbar daran, dass der alte Dateiname und der neue Dateiname im  
8 \.plcproj`-Diff  
9 sichtbar sind), MUSS Copilot den Typ-Benennungsverstoß auf **zwei** Arten  
10 melden:  
11  
12 1. An der Verwendungsstelle in einer **nicht umbenannten, modifizierten  
13 Datei**  
14 (z.B. `stc_FxControlPar.TcDUT`, `FxGroupManagement_V12.TcPOU`), sofern  
15 der  
16 Typ dort als neue Version des umbenannten Typs referenziert wird.  
17 Die Evidence MUSS die Zeile enthalten, die den Typnamen ohne korrektes Präfix  
18 zeigt.  
19 Der Befund MUSS explizit darauf hinweisen, dass die TYPE-Deklaration selbst  
verletzt ist.  
20  
21 2. In der \.plcproj`-Datei an der Zeile, die den neuen (prefix-losen) DUT-  
Dateinamen  
enthält, sofern diese Zeile im Diff als geänderter `+`-Eintrag erscheint.  
22  
23 **Copilot DARF NICHT** als einzigen Fundort die umbenannte DUT-Datei selbst  
angeben,  
da GitHub diese Kommentare als low-confidence unterdrückt.
```

Listing 2 – Anti suppression Klausel



Abbildung 6 – Anti Suppression Workflow

Damit werden Verstöße nicht nur an der Deklaration, sondern zusätzlich an der ersten Verwendung in einer nicht umbenannten Datei gemeldet. Die Wahrscheinlichkeit einer Unterdrückung durch GitHub sinkt dadurch deutlich. Dieser Ablauf ist in Abbildung 6 dargestellt. Nach dem Test zeigte sich, dass die ursprünglichen Kommentare weiterhin teilweise unterdrückt wurden, Verstöße jedoch zusätzlich an der ersten Verwendung der betroffenen Variablen als Fehler erschienen. Da diese Fundstellen nicht in umbenannten Dateien lagen, blieben die entsprechenden Meldungen sichtbar.

5.2.3 Iteration 3: Balance zwischen Präzision und Agilität

Nachdem die Review-Qualität durch präzise Instruktionen deutlich verbessert worden war, stellte Ralf Scheyerle ein neues Problem fest: Die stark restriktiven Anweisungen engen Copilot bei der täglichen Entwicklungsarbeit ein. Als Code-Assistent soll Copilot flexibel auf unterschiedliche Anfragen eingehen können; durch die strikten Reviewing-Klauseln verhielt sich das System jedoch auch außerhalb von Pull Requests zu rigide und verweigerte sinnvolle Vorschläge, die nicht unmittelbar einer Coding-Convention zugeordnet werden konnten.

Die Lösung bestand darin, die restriktivste Reviewing-Klausel aus den Instruktionen zu entfernen und die verbleibenden Anweisungen sprachlich zu öffnen, ohne die Kernaussagen zur Regelprüfung aufzugeben. Entscheidend war dabei, dass die Qualität der Befunde im Review-Betrieb erhalten blieb, während Copilot im Assistenzmodus wieder mit größerer Freiheit agieren konnte.

Die überarbeiteten TwinCat-Instructions wurden anschließend ins Übersichts-Repository übernommen und über den bestehenden Verteilungsprozess in alle relevanten Repositories propagiert (vgl. Abbildung 4). Damit gilt für Conventions und Instructions derselbe zentrale Aktualisierungsweg. Diese Balance zwischen Präzision im Review und Flexibilität als Assistent erwies sich als wichtige Voraussetzung für die spätere Akzeptanz im Team.

5.2.4 Iteration 4: Bestandscode-Analyse

Die bisherigen Iterationen hatten sich ausschließlich auf die Prüfung von Pull Requests konzentriert. Dabei wurde ein struktureller blinder Fleck sichtbar: Copilot wertet im PR-Review nur die veränderten Zeilen des Diffs aus. Code, der bereits vor Einführung der Conventions im Repository lag, insbesondere in Template-Projekten und Standard-Libraries, bleibt dadurch vollständig unkontrolliert. Regelverstöße im Bestandscode akkumulieren sich damit un bemerkt und können erst bei einer expliziten Änderung der betroffenen Stellen auffallen.

Um diese Lücke zu schließen, wurde in Zusammenarbeit mit Ralf Scheyerle ein spezialisierter Agent entwickelt, der eine vollständige Analyse ausgewählter Repositories durchführt, unabhängig von Pull Requests. Der Agent bezieht das gesamte Repository als Kontext ein, prüft alle relevanten Dateien gegen die aktuellen Coding-Conventions und legt die Befunde strukturiert ab. Im

Anschluss an die Entwicklung wurde der Agent unmittelbar auf Template-Projekte und Standard-Libraries angewendet.

Diese Ergänzung schließt die methodische Lücke der reinen PR-basierten Prüfung. Beide Ansätze ergänzen sich dabei: Pull-Request-Reviews sichern den Entwicklungsfluss kontinuierlich ab, während der Bestandscode-Agent dafür sorgt, dass auch historisch gewachsene Regelverstöße systematisch erfasst und schrittweise in den definierten Standard überführt werden können, ohne den laufenden Betrieb zu unterbrechen.

5.2.5 Zusammenfassende Bewertung der Iterationen

Zur konsistenten Einordnung werden die vier Iterationen entlang der in Tabelle 2 definierten Kriterien zusammengeführt.

Iteration	Ziel der Anpassung	Befundpräzision	Unterdrückungsanfälligkeit
1	Präzisere und referenzierbare Conventions	deutlich verbessert	unverändert
2	Sichtbarkeit bei Umbenennungen erhöhen	stabil	verbessert, aber nicht eliminiert
3	Review-Präzision mit Assistenz-Flexibilität balancieren	hoch	unverändert
4	Bestandscode außerhalb von PR-Diffs erfassen	hoch bei Vollanalyse	nicht PR-diff-abhängig

Tabelle 6 – Iterationsvergleich entlang zentraler Evaluationsmetriken

In Summe zeigt Tabelle 6 eine klare Reifung des Ansatzes: Die fachliche Präzision steigt über die Iterationen konsistent an, während plattformbedingte Einschränkungen gezielt durch Prozessmaßnahmen kompensiert werden.

5.3 Zwischenfazit zur Evaluation

Die vier Iterationen zeigen einen klaren Entwicklungspfad. Ausgangspunkt war ein System, das grundsätzlich funktionsfähig, aber in seiner Prüftiefe unzureichend war. Der entscheidende Hebel lag nicht in der KI-Technologie selbst, sondern in der Qualität der zugrunde liegenden Conventions und Instruktionen: Erst präzise, eindeutig referenzierbare Regeln ermöglichten reproduzierbare und fachlich verwertbare Befunde.

Die zweite Iteration machte deutlich, dass die Qualität eines Reviews nicht allein von Copilot abhängt, sondern auch von der Plattform GitHub. Das Unterdrücken von Befunden bei umbenannten Dateien ist ein inhärentes Verhalten der GitHub-Diff-Auswertung und lässt sich nicht durch Instruktionsänderungen allein beheben. Die entwickelte Anti-Suppression-Strategie adressiert dieses Problem auf Prozessebene, indem Befunde an zusätzlichen, nicht betroffenen Fundstellen platziert werden.

Iteration 3 belegt, dass Präzision und Flexibilität keine Gegensätze sind, wenn Instruktionen zielgerichtet formuliert werden. Die Trennung zwischen verbindlichen Review-Verhaltensregeln und allgemeinen Assistenzfunktionen erlaubt es, beide Anforderungen gleichzeitig zu erfüllen.

Die vierte Iteration schließlich erweitert den Anwendungsbereich über den Pull-Request-Prozess hinaus. Damit ist die Qualitätssicherung nicht länger auf den Moment der Codeänderung beschränkt, sondern kann retrospektiv auf den gesamten Bestand angewendet werden. Für die praktische Einführung in der O-SW bedeutet das, dass Bestandsprojekte und Bibliotheken systematisch nachbereitet werden können, ohne den laufenden Entwicklungsbetrieb zu unterbrechen.

In der Gesamtschau ist festzuhalten: KI-gestützte Code-Reviews sind ein wirksames Werkzeug zur Qualitätssicherung im TwinCat-Umfeld, setzen aber eine sorgfältige Pflege der Conventions und Instruktionen voraus. Die erarbeiteten Prinzipien Präzision, Referenzierbarkeit, gezielte Unterdrückungsvermeidung und Bestandsbereinigung bilden die Grundlage für den stabilen Einsatz im Pilotprojekt.



Entwicklungsumgebung

Die Entwicklungsumgebung kombiniert VS Code und TwinCat XAE Shell. VS Code lässt sich über Extensions flexibel an Projektanforderungen anpassen, ist Open Source und ressourcenschonend.

Entscheidend ist jedoch nicht nur die Tool-Auswahl, sondern ein einheitlicher Ablauf. Daher werden Dateiverwaltung, SPS-Programmierung und GitHub-Verwaltung im Folgenden prozessorientiert beschrieben.

6.1 Dateiverwaltung

Die Dateiverwaltung erfolgte bisher überwiegend im Windows Explorer. Künftig wird sie in VS Code gebündelt, um Projektstruktur, Textartefakte und Versionsstatus in einer Oberfläche zu führen.

Soll-Ablauf:

- Projekt lokal öffnen und Verzeichnisse im VS Code Explorer verwalten
- Relevante Textartefakte (z. B. XML-Dateien) direkt im selben Arbeitsfenster bearbeiten
- Änderungen im Source-Control-Bereich prüfen, dann committen

Der Wechsel zwischen Explorer, separatem Git-Client und Editor entfällt. Das erhöht Effizienz und Nachvollziehbarkeit.

Zusätzlich verbessert die Bündelung in VS Code die Einarbeitung neuer Mitarbeitender: Wiederkehrende Aufgaben wie Dateivergleich, Suche, Commit-Vorbereitung und PR-Navigation folgen einem einheitlichen Bedienmuster.

Dadurch sinkt die Abhängigkeit von individuellen Tool-Präferenzen einzelner Teammitglieder.

6.2 SPS-Programmierung

TwinCat-Programme liegen XML-basiert vor und lassen sich nicht sinnvoll in beliebigen Editoren bearbeiten. Auch Kompilierung, Steuerungs-Upload und Debugging erfordern Beckhoff-spezifische Werkzeuge.

Daher erfolgt die SPS-Programmierung in der TwinCat XAE Shell statt in Visual Studio. Die XAE Shell ist kostenfrei, in der Praxis oft performanter und erhält neue TwinCat-Versionen in der Regel zuerst.

Rollenverteilung der Werkzeuge:

- VS Code: Projektorganisation, Versionsverwaltung, Reviews, begleitende Textartefakte
- TwinCat XAE Shell: SPS-Programmierung und Bearbeitung der projektspezifischen XML-Inhalte

Diese Trennung reduziert Tool-Konflikte. Fachliche Änderungen entstehen in der XAE Shell, kollaborative Schritte laufen in VS Code und GitHub.

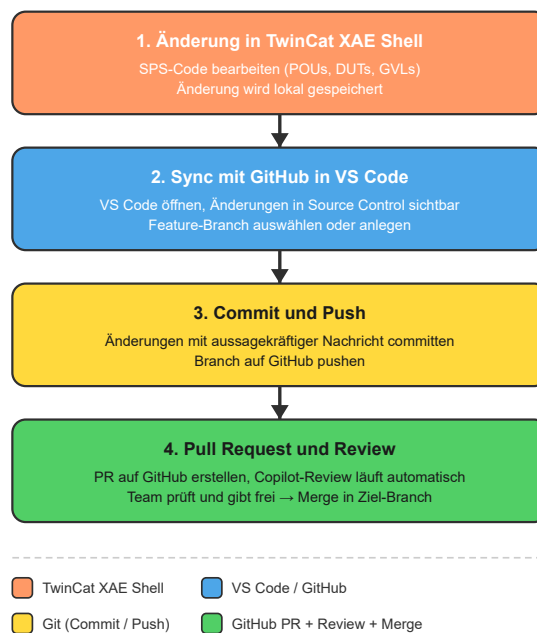


Abbildung 7 – Workflow: Änderung in XAE Shell bis Pull Request

6.3 GitHub Verwaltung

Die GitHub-Verwaltung erfolgte bisher vor allem über Sourcetree oder GitHub Desktop. In VS Code steht jedoch bereits eine integrierte Oberfläche für pull, push, commit, checkout, Branch-Auswahl, Arbeitsbaum und Änderungsverwaltung bereit.

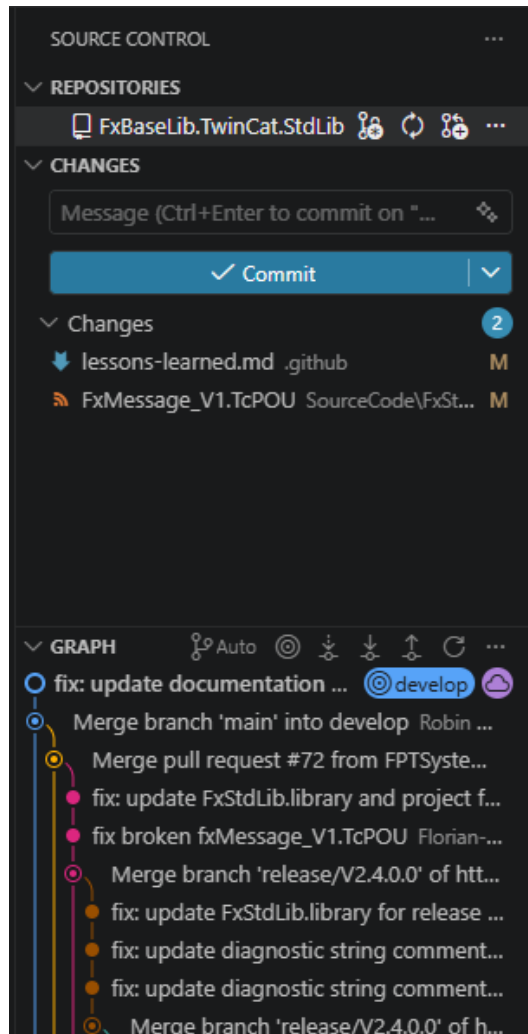


Abbildung 8 – VS-Code Source Verwaltung

Das Interface ist über die Seitenleiste jederzeit erreichbar (vgl. Abbildung 8). Für Issues, Actions und Pull Requests ergänzen die empfohlenen Extensions „GitHub Pull Requests“ und „GitHub Actions“ den Funktionsumfang.

Durchgängiger Tagesablauf:

- Lokalen Stand synchronisieren (fetch/pull) und auf den Arbeitsbranch wechseln

- Änderung umsetzen und den Arbeitsbaum fortlaufend prüfen
- Commit und Push in VS Code, danach Pull Request erstellen oder aktualisieren
- Review und Freigabe über die Integration, anschließend Merge in den Zielbranch

Diese integrierte Vorgehensweise reduziert Medienbrüche und stärkt Pull-Request-basierte Freigaben sowie nachvollziehbare Änderungsverläufe.

Für den Tagesbetrieb bedeutet das eine klare Trennung von fachlicher Umsetzung und Freigabeschritten: Änderungen entstehen in den jeweiligen Arbeitswerkzeugen, werden aber zentral in GitHub-konformen Abläufen dokumentiert, geprüft und integriert. Insbesondere bei parallel laufenden Arbeitspaketen steigt dadurch die Planbarkeit der Merge-Reihenfolge.



Pilotierung im Projekt

Um die praxisnahe Integration der eingeführten Qualitätssicherungsprotokolle zu bewerten, wurden diese in ein Projekt eingebracht, das sich noch in der Vorinbetriebnahme befand. Dazu wurde mit den Projektprogrammierern David Kromer und Alexandru Trusi ein Termin angesetzt, in dem die geplanten Methoden gemeinsam besprochen wurden. Insbesondere die Branching-Strategien wurden positiv bewertet, da beide Programmierer die in Abschnitt 3.1 beschriebenen Herausforderungen bereits aus ihrem Arbeitsalltag kannten.

Alexandru brachte zusätzlich den Vorschlag ein, neben Feature-Banches pro Entwickler jeweils einen eigenen Develop-Branch zu nutzen. Dadurch kann zunächst unabhängig gearbeitet werden, bevor stabile Zwischenstände per Pull Request in develop überführt werden. Ergänzend können nutzerspezifische Feature-Banches für die Umsetzung konkreter Funktionen angelegt werden. Die Benennung könnte beispielsweise wie folgt aussehen:

- user/develop
- user/feature/NeuesFeature

Beide Programmierer signalisierten außerdem Offenheit gegenüber Copilot, sofern die Review-Ergebnisse konsistent und verlässlich bleiben.

In einer anschließenden Abstimmung zu den Meeting-Ergebnissen merkte Robin Hettel an, dass statt nutzerbezogener Branches eher anlagenbereichsbezogene Branches sinnvoll seien. Dadurch könnten Programmierer einzelne Bereiche zeitweise „reservieren“ und dort konzentriert arbeiten. Gleichzeitig würde dieser Ansatz helfen, keine gesonderten Richtlinien nur für die Anlagenprogrammierung einzuführen, sondern einen unternehmensweit einheitlichen Standard beizubehalten.

Nach der Formulierung dieser Richtlinien startete das Projekt in seine erste Phase, die virtuelle Inbetriebnahme. In dieser Phase werden die Roboter in einer Simulation angelegt und bereits grundlegend programmiert.

7.1 Umsetzung im Pilotprojekt

7.1.1 Branching Strategien

Die Branching-Strategie wurde auf Basis des angepassten Git-Flow-Modells umgesetzt. Dabei wurden die Aufgaben der einzelnen Branches klar abgegrenzt und der Merge-Prozess über Pull Requests verbindlich standardisiert. Ergänzend wurden Benennungsregeln für Feature-, Bugfix- und Release-Banches festgelegt, um eine hohe Nachvollziehbarkeit und einheitliche Abläufe sicherzustellen.

ERFAHRUNGEN UND PROBLEME

Die Einführung der Branching-Strategie verlief im Projektverlauf insgesamt stabil. Die Projektprogrammierer David Kromer und Alexandru Trusi hielten die Branch-Struktur im Wesentlichen ein; Feature-Banches wurden für neue Funktionen angelegt und über Pull Requests in den gemeinsamen Develop-Branch integriert. Merge-Konflikte traten auf, ließen sich jedoch durch die klare Branch-Abgrenzung schneller lokalisieren und auflösen als in vorherigen Projekten ohne einheitliche Konventionen.

Als besonders positiv wurde festgehalten, dass der Pull-Request-Übergang als natürlicher Kontrollpunkt wirkt: Bevor Code in Develop einfließt, findet eine explizite Prüfung statt. Dadurch werden Fehler und Inkonsistenzen früher sichtbar, bevor sie sich in nachgelagerten Integrationsphasen auswirken.

Als Herausforderung zeigte sich der anfängliche Mehraufwand in der Einarbeitungsphase. Das konsequente Anlegen und Benennen von Branches sowie das Vorbereiten reviewfähiger Pull Requests erfordert Disziplin und erhöht zunächst den Zeitaufwand pro Änderung. Dieser Aufwand relativiert sich mit zunehmender Routine; er ist als Investition in die langfristige Transparenz und Nachvollziehbarkeit zu verstehen.

Im Projektverlauf wurde außerdem festgestellt, dass die Coding-Conventions auskommentierten Code als Regelverletzung werten. Da vorübergehend de-

aktivierter Code in der Entwicklungsphase jedoch ein reguläres Mittel ist, wurde in Absprache mit dem Team eine pragmatische Regel eingeführt: In develop darf auskommentierter Code verbleiben, sofern er mit einem `{warning 'ToDo: '}`-Tag kenntlich gemacht wird. In Branches auf main-Richtung bleibt die ursprüngliche Regel weiterhin gültig.

Zusätzlich wurde identifiziert, dass `.tmc`-Dateien (TwinCat-Maschinenkonfigurationen) in die `.gitignore` aufgenommen werden sollten. Diese Dateien enthalten maschinenspezifische Einstellungen, die nicht projektübergreifend versioniert werden sollen und bei fehlendem Eintrag zu unerwünschten Konflikten im Repository führen können.

7.1.2 Copilot Code-Review

Das Copilot-gestützte Code-Review läuft analog zum Test-Repository. Jeder PR zu main wird automatisch gegen die Coding-Conventions geprüft; Befunde integrieren sich direkt in den Freigabeprozess.

ERFAHRUNGEN UND PROBLEME

Das Projekt lief regulär mit einem Unterschied: Jeder PR zu main löste automatisch einen Copilot-Review aus.

Nach dem ersten Meilenstein (Virtuelle Inbetriebnahme) zog Alexandru folgende Bilanz:

- Rechtschreibfehler: Zuverlässig erkannt, automatischer Resolve funktioniert
- Falsch-Positive: Manchmal Meldungen für Variablen, die in FUP tatsächlich verwendet werden
- Schweregrad-Einstufung: Copilot überschätzt manchmal die Kritikalität von Fehlern
- Syntaxfehler in Vorschlägen: Strukturelle Verbesserungen erfordern teils manuelle Nachprüfung
- Strukturelle Befunde: Copilot erkannte einen Strukturfehler im Code korrekt; der zugehörige automatische Verbesserungsvorschlag war jedoch syntaktisch fehlerhaft und hätte nicht direkt übernommen werden dürfen. Als Konsequenz wurde die Funktion „Commit Suggestion“, deaktiviert, um ungeprüfte automatische Korrekturen zu verhindern.

- **Kommentarvorschläge:** Bei Rechtschreib- und Formulierungsfehlern in Kommentaren lieferte Copilot konsistent gute und direkt anwendbare Verbesserungsvorschläge.

Diese Rückmeldung deckt sich mit den Beobachtungen aus der Evaluation: Copilot liefert einen messbaren Mehrwert bei formalen und sprachlichen Qualitätsaspekten, benötigt bei domänenspezifischen SPS-Kontexten jedoch weiterhin menschliche Plausibilisierung. Besonders bei FUP-bezogenen Verwendungsstellen ist der semantische Kontext nicht immer vollständig ableitbar, was zu vereinzelt Falsch-Positiven führt.

Für den Pilotbetrieb wurde deshalb ein pragmatischer Umgang festgelegt: Kritische oder strukturelle Empfehlungen werden als Prüfhinweis behandelt und erst nach manueller Verifikation umgesetzt; klar nachvollziehbare Formalverstöße können direkt korrigiert werden. Dadurch bleibt der Nutzen des automatisierten Reviews erhalten, ohne ungeprüfte Änderungen in den Codebestand zu übernehmen. Alexandru beurteilte den Gesamtnutzen nach dem ersten Meilenstein als positiv: Der Review-Aufwand für formale Aspekte sank spürbar, während der fachlich inhaltliche Review weiterhin manuell bleibt. Dieses Zusammenspiel aus automatisierter Formalkontrolle und menschlicher Fachkontrolle erwies sich als praxistaugliches Modell für die weitere Projektarbeit.

Zur semiquantitativen Einordnung wurde für den Verlauf der virtuellen Inbetriebnahme im Projekt GIRA folgende Stichprobe betrachtet (n = 40 Copilot-Befunde):

Kategorie	Anzahl	Anteil
Korrekt erkannte Befunde	25	62,5 %
Manuelle Prüfung erforderlich	13	32,5 %
Falsch-Positive	2	5,0 %
Summe	40	100 %

Tabelle 7 – Semiquantitative Befundauswertung (GIRA, virtuelle Inbetriebnahme)

Die 2 Falsch-Positiven traten überwiegend in Situationen auf, in denen Copilot FUP-Kontexte in TwinCat nicht eindeutig interpretieren konnte. Die 13 manuell zu prüfenden Befunde entstanden bis auf 2 Fälle durch temporär

auskommentierten Code oder projektbedingte Workarounds während der Entwicklungsphase.

Für diesen Fall wurde eine verbindliche Teamregel festgelegt: Auskommentierter Code ist in der Entwicklungsphase zulässig, sofern direkt darüber ein `{warning 'ToDo:'}`-Hinweis steht, dass der betroffene Abschnitt vor Release entweder entfernt oder wieder in Betrieb genommen werden muss. Dieser Hinweis sorgt innerhalb der XAE Shell für eine Warnung im Fehlerfenster. Dies wurde im Anschluss auch in den Copilot Anweisungen eingearbeitet, dass hier bei einem PR in develop kein Fehler angemerkt wird.

Ergänzend wurde aus vergleichbaren Projekten ähnlicher Größenordnung eine explizite Schätzung dokumentiert: Ohne den eingeführten Review-Ansatz verbleiben schätzungsweise rund 100 kleinere Benennungs- und Rechtschreibfehler (Variablen und Kommentare) zunächst unentdeckt und werden häufig erst in späteren Wartungsphasen sichtbar. Diese Zahl ist als Erfahrungswert zu verstehen und variiert zwischen Entwicklern und Projekten teils deutlich. Im neuen Ablauf wurden solche formalen Fehler im betrachteten Projektabschnitt nicht mehr als spätere Wartungsbefunde festgestellt, da sie bereits im PR-Prozess adressiert wurden. Für den ausgewerteten Projektabschnitt und die betrachteten PRs kann mit hoher Sicherheit gesagt werden, dass durch die neuen Review-Abläufe keine relevanten Strukturfehler mehr verblieben sind, die unter bestimmten und zuvor nicht beachteten Umständen zu Fehlern geführt hätten.

Zeitlich zeigt sich im Pilotbetrieb eine Verschiebung des Aufwands: Reviews auf `feature/*` -> `develop` dauern tendenziell länger als zuvor, da sie vorher nicht vorhanden waren, während Reviews auf `develop` -> `main` deutlich entlastet werden und statt mehrerer Stunden in der Regel unter einer Stunde abgeschlossen werden können.



Projektmanagement mit ZenHub

ZenHub ist ein webbasiertes Projektmanagement-Tool für Softwareentwicklung, das direkt in GitHub integriert ist (vgl. [7]). Der zentrale Vorteil liegt darin, dass Aufgabenplanung und Quellcodeverwaltung in einer gemeinsamen Oberfläche zusammengeführt werden. Medienbrüche zwischen Ticket-System, E-Mail-Kommunikation und Repository entfallen dadurch vollständig: Entwickler können Issues direkt mit Branches und Pull Requests verknüpfen, ohne zwischen verschiedenen Plattformen wechseln zu müssen.

Für die O-SW ist ZenHub ein strategisches Ergänzungselement zu den technischen Prozessverbesserungen. Während Branching-Strategien und Code-Reviews die technische Qualität der Entwicklungsartefakte sichern, adressiert ZenHub die organisatorische Seite: Welche Aufgabe hat welche Priorität? Wer ist verantwortlich? Wann ist ein Release geplant? Diese Fragen lassen sich ohne ein zentrales Planungswerkzeug nur schwer transparent und verbindlich beantworten.

ZenHub stellt dafür vier funktionale Bausteine bereit. Boards ermöglichen die operative Steuerung des Tagesgeschäfts, indem Issues per Drag-and-Drop durch konfigurierbare Spalten bewegt werden. Epics bündeln thematisch verwandte Issues zu übergeordneten Arbeitspaketen und ermöglichen damit eine strukturierte Fortschrittsverfolgung auf Funktions- oder Meilensteinebene. Roadmaps stellen die zeitliche Planung über Iterationen und Releases hinweg dar und machen Abhängigkeiten und kritische Pfade sichtbar. Automatisierungen schließlich reduzieren manuellen Verwaltungsaufwand, indem

Statuswechsel regelbasiert ausgelöst werden, etwa wenn ein zugehöriger Pull Request gemergt wird.

Die Kombination dieser Bausteine schafft eine zentrale, nachvollziehbare Planungs- und Kommunikationsstruktur, die unabhängig von Teamgröße skaliert.

8.1 E-SW Prozess als Referenz

In der E-SW steuert ZenHub primär die Planung und Durchführung von FlexOp-Releases. Der etablierte Ablauf beginnt mit der Erfassung und Vorqualifizierung von Issues durch die Entwickler. Teamleiter Simon Würzer übernimmt anschließend die Priorisierung anhand von Dringlichkeit und fachlicher Bedeutung. Priorisierte Tickets werden konkreten Entwicklern zugeordnet und fachlich eingewiesen. Größere Vorhaben, die mehrere zusammenhängende Arbeitsschritte umfassen, werden als Epics angelegt und in einzelne Issues unterteilt. Release-Zeitpunkte werden in der Roadmap hinterlegt und den zugehörigen Issues zugeordnet, sodass jederzeit erkennbar ist, welche Pakete für welchen Release eingeplant sind.

Im Tagesgeschäft hat sich dabei besonders die Kombination aus Epic-Struktur und Board-Status bewährt. Auch wenn kurzfristig umpriorisiert werden muss, bleibt für alle Beteiligten sichtbar, welche Arbeitspakete zu welchem Release-Ziel gehören und welche Abhängigkeiten noch offen sind. Dadurch kann die Teamleitung kurzfristige Entscheidungen treffen, ohne den Gesamtfortschritt aus dem Blick zu verlieren. Die direkte Verknüpfung von Issues mit Branch-Namen, beispielsweise `feature/Fx.FlexOP-1640_SSI-Packagerstellung`, stellt außerdem sicher, dass technische Umsetzung und Planungsstand jederzeit konsistent bleiben.

8.2 Übertragung auf die O-SW

Ziel der ZenHub-Einführung in der O-SW ist es, die bisher mündlich und per E-Mail koordinierten Planungsprozesse durch einen standardisierten, nachvollziehbaren Ablauf zu ersetzen. Dies ist insbesondere dann relevant, wenn kurzfristige Versionsänderungen an FlexOp Auswirkungen auf SPS-Projekte haben: Abhängigkeiten werden früh sichtbar, der Abstimmungsaufwand sinkt, und Priorisierungsentscheidungen lassen sich transparent dokumentieren.

Das konkrete Vorgehen sieht vor, dass Issues für alle relevanten Arbeitspakete in GitHub angelegt, priorisiert und Verantwortlichen zugeordnet werden. Größere Vorhaben, etwa die Weiterentwicklung einer Standardbibliothek oder die Vorbereitung eines Template-Releases, werden als Epics strukturiert und in handhabbare Einzelissues unterteilt. Zeitkritische Pakete werden Meilensteinen zugeordnet, um die Release-Abhängigkeiten zwischen O-SW und E-SW sichtbar zu machen.

Für die Board-Konfiguration wurden Spalten nach dem Muster Backlog → Planned → In Progress → Review → Done festgelegt. Diese Aufteilung spiegelt den tatsächlichen Arbeitsfluss wider: Ein Issue wandert von der Erfassung über die aktive Bearbeitung durch die Code-Review-Phase bis zur abgeschlossenen Integration. Die Rollen wurden klar definiert: Priorisierung und Release-Freigabe liegen bei der Teamleitung, die operative Aufgabenverwaltung bei den Entwicklern.

ZenHub fungiert damit nicht als isoliertes Ticket-System, sondern als Verbindungselement zwischen technischer Umsetzung, Branching-Prozess und Release-Kommunikation. Issues, Pull Requests und Branches lassen sich in einem durchgängigen Ablauf verknüpfen, sodass Planungs- und Entwicklungsstand jederzeit konsistent und für alle Beteiligten einsehbar bleiben. Für die O-SW ist entscheidend, dass ZenHub nicht als administrativer Overhead verstanden wird, sondern als operatives Steuerungsinstrument, das den bestehenden GitHub-Workflow gezielt ergänzt.

8.3 Zwischenfazit zum Projektmanagement

Erkenntnis: Die Kopplung von ZenHub mit Branching- und Review-Prozessen verbessert die organisatorische Steuerbarkeit, weil Planungs- und Entwicklungszustände auf denselben Artefakten aufbauen.

Implikation: Besonders wirksam ist die Einführung dann, wenn Rollen für Priorisierung, Bearbeitung und Freigabe verbindlich definiert sind und Board-Status diszipliniert gepflegt werden.



Schulung und Wissenssicherung

Damit die Methoden wirksam und konsistent greifen, entstehen strukturierte Schulungsunterlagen. Sie unterstützen das Onboarding neuer Mitarbeitender und etablieren abteilungsweit einheitliche Standards.

9.1 Bedarfsermittlung

Im Vorfeld der Schulungsplanung fand ein Austausch mit dem Mitarbeiter Matthias Scheuring statt, der repräsentativ für die Zielgruppe der Schulungsmaßnahmen steht: erfahrene SPS-Programmierer mit Projektberührung, aber geringer Vorerfahrung im Bereich Git und GitHub.

Das Gespräch ergab drei zentrale Einstiegshürden. Das Grundverständnis für Git und GitHub war nicht ausreichend vorhanden, was dazu führte, dass die Werkzeuge nicht konsequent eingesetzt wurden. In der Praxis äußerte sich das in wiederkehrenden Rückfragen, wie Commits korrekt durchgeführt werden. Das ist ein deutlicher Hinweis darauf, dass die grundlegende Mechanik noch nicht sicher verankert war.

Die Branch-Mechanik bereitete anfangs ebenfalls Schwierigkeiten. Ohne ein klares Verständnis davon, wozu Branches dienen und wie sie sich voneinander abgrenzen, wurden Commits direkt in main vorgenommen. Dies führte zu Konflikten im Code und im Team, wie in Abschnitt 3.1.1 beschrieben.

Als dritte Hürde fehlte der Unternehmenskontext: Es war unklar, warum bestimmte Prozesse so gestaltet sind, welche Konventionen für die O-SW gelten und wie der eigene Arbeitsschritt in den Gesamtprozess eingebettet ist. Ohne dieses Prozessverständnis werden selbst bekannte Werkzeuge nicht korrekt angewendet, weil die Konsequenzen von Abweichungen nicht einschätzbar sind.

Aus dieser Analyse ergeben sich drei Schulungsbausteine, die aufeinander aufbauen und sowohl technische Kenntnisse als auch prozessuales Verständnis vermitteln.

9.2 Schulungsumgebung: Evaluierte Optionen

Bei der Wahl des Schulungsformats wurden mehrere externe und interne Optionen gegenübergestellt. Microsoft Learn bietet etablierte, gut strukturierte GitHub-Kurse, lässt sich jedoch nicht an die fpt-spezifischen Prozesse anpassen. Teile der Inhalte beschreiben Workflows, die von den in dieser Arbeit eingeführten Konventionen abweichen, und könnten damit zu Missverständnissen führen.

GitHub Skills stellt hochwertige, interaktive Lernpfade bereit, ist jedoch stark auf einen workspace-basierten Ansatz ausgerichtet. Die Übertragbarkeit auf den TwinCat- und WorkVisual-Kontext der O-SW ist begrenzt, da diese Werkzeuge in den Inhalten keine Berücksichtigung finden.

YouTube bietet umfangreiche und didaktisch vielfältige Inhalte zu Git und GitHub. Für den grundlegenden Einstieg sind einzelne Videos geeignet, jedoch fehlen O-SW-spezifische Prozesse vollständig. YouTube eignet sich deshalb nur als ergänzendes Medium, nicht als primäres Schulungsinstrument.

Die eigene Dokumentation im fpt-internen Wiki erweist sich als die geeignetste Lösung. Inhalte können beliebig an die tatsächlich eingesetzten Prozesse angepasst, multimedial ergänzt und jederzeit aktualisiert werden. Die Unterlagen sind für alle Mitarbeitenden zugänglich und können als langfristiges Nachschlagewerk genutzt werden.

Die gewählte Strategie kombiniert Wiki-basierte Dokumentation mit ergänzenden Präsentationen für Einweisungsgespräche und ausgewählten YouTube-Videos für den allgemeinen Einstieg.

9.3 Umsetzung der Dokumentation

9.3.1 Git und GitHub Grundlagen

Die Dokumentation der Git- und GitHub-Grundlagen bildet den Einstieg für neue Mitarbeitende und Auszubildende. Behandelt werden Versionsverwaltung, Branches und Pull Requests im Unternehmenskontext. Sie dient sowohl zum Onboarding als auch als dauerhaftes Nachschlagewerk.

Inhaltlich deckt der Baustein die Grundkonzepte von Git ab: die Unterscheidung zwischen Working Directory, Staging Area und Repository sowie die Bedeutung von Commits als atomare Änderungseinheiten. Der typische Team-Workflow mit den Schritten klonen, ändern, committen, pushen und Pull Request erstellen wird schrittweise erläutert. Ergänzend werden Branches, Merges und Konfliktlösung behandelt, da diese Konzepte erfahrungsgemäß die häufigsten Verständnisprobleme verursachen.

Das Format kombiniert eine schriftliche Referenz im fpt-Wiki, einen Link zu einer begleitenden Präsentation und ein einführendes Video-Tutorial. Dieses mehrstufige Lernangebot ermöglicht es, je nach Vorwissen unterschiedliche Einstiegspunkte zu wählen. Der Rollout erfolgt über Wiki-Publikation, anschließende Einweisungsgespräche, eine Feedbackrunde und abschließende Integration der fpt-spezifischen Vorgaben zu Branching und Reviews.

Nach Abschluss des Bausteins können die Teilnehmenden Git- und GitHub-Grundkonzepte erklären und voneinander abgrenzen, den Standard-Workflow sicher durchführen, Branches und Merges korrekt handhaben sowie typische Probleme erkennen und eigenständig lösen.

9.3.2 Branching Strategien

Im Schulungsbaustein zu Branching-Strategien wird das in dieser Arbeit eingeführte Vorgehen praxisnah vermittelt. Der Schwerpunkt liegt auf der eindeutigen Zuordnung von Branch-Typen, der strukturierten Nutzung von Pull Requests sowie der nachvollziehbaren Integration in develop und main. Ziel ist eine konsistente Anwendung der festgelegten Regeln im Tagesgeschäft.

Die Unterlage baut inhaltlich auf den Git- und GitHub-Grundlagen auf und überführt diese in den konkreten Teamprozess der [O-SW](#). Zunächst werden Zweck und Verantwortlichkeiten der verwendeten Branch-Typen systema-

tisch erläutert (main, develop, feature/*, bugfix/*, release/*, hotfix/*). Darauf aufbauend wird der verbindliche Ablauf vom Anlegen eines Arbeitsbranches bis zum Merge über Pull Requests dargestellt. Ein besonderes Gewicht liegt auf den in der Abteilung eingeführten Regeln „keine direkte Arbeit auf main“ und „nach Möglichkeit keine direkte Arbeit auf develop“, um ungeprüfte Integrationen zu vermeiden.

Ergänzend werden Benennungsstandards und typische Fehlermuster behandelt. Dazu zählen unter anderem uneinheitliche Branch-Namen, fehlende Rückführung von Hotfixes in develop sowie zu große oder fachlich gemischte Pull Requests. Die Teilnehmenden erhalten hierfür klare Leitlinien zur Benennung, zur inhaltlichen Abgrenzung von Commits und zur Vorbereitung reviewfähiger Pull Requests.

Zur didaktischen Strukturierung umfasst der Baustein drei Ebenen: erstens eine kompakte Prozessdarstellung als Referenz, zweitens angeleitete Schritt-für-Schritt-Abläufe für wiederkehrende Standardsituationen (Feature, Bugfix, Release, Hotfix) und drittens kurze Übungsaufgaben mit typischen Teamkonstellationen. Damit wird sichergestellt, dass nicht nur Begriffe bekannt sind, sondern der Ablauf im Projektalltag sicher angewendet werden kann.

Lernziele des Bausteins:

- Die Teilnehmenden können die Funktion der Branch-Typen im angepassten Git-Flow-Modell fachlich korrekt erklären und voneinander abgrenzen.
- Die Teilnehmenden können für eine gegebene Aufgabe den passenden Branch-Typ auswählen und regelkonform benennen.
- Die Teilnehmenden können einen vollständigen Standardablauf von der Änderung bis zum Merge über Pull Request durchführen und begründen.
- Die Teilnehmenden können typische Prozessfehler erkennen (z. B. falscher Zielbranch, fehlende Rückführung von Hotfixes) und geeignete Korrekturen ableiten.

9.3.3 Entwicklungsumgebung

Der Schulungsbaustein zur Entwicklungsumgebung vermittelt den sicheren Umgang mit VS Code im täglichen GitHub-Workflow. Ausgangspunkt ist eine PowerPoint-Präsentation, die das Grundverständnis der Oberfläche aufbaut; die dazugehörige Wiki-Dokumentation dient als dauerhaftes Referenzdokument.

Inhaltlich umfasst der Baustein die Installation und Einrichtung von VS Code inklusive der empfohlenen Extensions sowie die Konfiguration eines Workspace. Der tägliche Workflow wird vollständig abgebildet: von fetch und pull über den Branch-Wechsel und die Bearbeitung bis hin zu Commit, Push und Pull-Request-Erstellung. Ein besonderes Augenmerk liegt auf der Integration der TwinCat XAE Shell, da hier das Zusammenspiel beider Werkzeuge für SPS-Programmierer nicht selbstverständlich ist. Der typische Ablauf wird als durchgängige Schrittfolge dargestellt: Änderung in der XAE Shell vornehmen, in VS Code committen und pushen. Abschließend wird der KI-Einsatz über den Copilot-Workflow thematisiert, inklusive der Erwartungshaltung an automatische Befunde und der Notwendigkeit kritischer Bewertung der Ergebnisse.

Didaktisch folgt der Baustein dem gleichen Muster wie die übrigen Schulungskapitel: kompakte Einführung, standardisierte Schrittfolgen, Best Practices und typische Fehler. Nach Abschluss können die Teilnehmenden das VS-Code-Setup selbstständig durchführen, den täglichen GitHub-Workflow sicher anwenden, die XAE Shell korrekt als Komplement einsetzen und KI-Hinweise kritisch als Entscheidungshilfe bewerten. Damit werden nicht nur Bedienkenntnisse vermittelt, sondern ein verbindliches Verhaltensmodell für den Teamalltag etabliert.

9.4 Zwischenfazit zur Schulung und Wissenssicherung

Erkenntnis: Die Wirksamkeit der technischen Maßnahmen hängt wesentlich davon ab, ob Prozesswissen, Tool-Bedienung und Rollenverständnis gemeinsam vermittelt werden.

Implikation: Ein mehrstufiges Lernangebot aus Wiki, Präsentation und betreuter Praxis reduziert Einstiegshürden und stabilisiert die Anwendung der Standards im Tagesgeschäft.

10

Einführung im Team

Die Einführung der neuen Vorgehensweise erfolgt stufenweise, um die operative Projektarbeit nicht zu unterbrechen. Zunächst werden die Kernprozesse Branching, Pull-Request-Freigabe und ZenHub-Board-Nutzung in einer kompakten Basisschulung vermittelt. Diese Schulung richtet sich an alle Beteiligten der O-SW und kombiniert Prozessübersicht mit kurzen, praxisnahen Übungen aus dem tatsächlichen Arbeitsumfeld.

Im zweiten Schritt wird die Anwendung in laufenden Projekten durch begleitete Praxisphasen abgesichert. Dabei werden in definierten Zeitfenstern reale Arbeitspakete nach dem neuen Standard durchgeführt und anschließend gemeinsam ausgewertet. Ziel ist es, Unsicherheiten früh sichtbar zu machen und die Richtlinien dort nachzuschärfen, wo sie im Alltag noch zu Interpretationsspielraum führen.

Zur nachhaltigen Verankerung wird ergänzend ein leicht zugängliches Wissenspaket im internen Wiki bereitgestellt. Dieses umfasst Prozessbeschreibungen, Namenskonventionen, Standardabläufe für typische Branching-Situationen sowie Leitlinien zur Bewertung von Copilot-Hinweisen. Damit steht sowohl für Neueinsteiger als auch für erfahrene Mitarbeitende eine einheitliche Referenz zur Verfügung.

Die Wirksamkeit der Einführung wird über regelmäßige Kurz-Reviews im Team überprüft. In diesen Terminen werden wiederkehrende Probleme gesammelt, Verbesserungsmaßnahmen beschlossen und dokumentiert. So entsteht ein kontinuierlicher Verbesserungsprozess, der technische Qualität und organisatorische Umsetzbarkeit gleichermaßen berücksichtigt.



Diskussion, Limitationen und Wirtschaftlichkeit

11.1 Einordnung der Ergebnisse

Die Ergebnisse zeigen, dass die Kombination aus Branching-Regeln, Copilot-Review und integriertem Team-Management als Gesamtsystem wirkt. Einzelmaßnahmen erzeugen bereits lokale Verbesserungen, erreichen jedoch erst im Zusammenspiel eine stabile Wirkung auf Nachvollziehbarkeit, Qualitätskontrolle und Planbarkeit. Insbesondere die Kopplung von PR-Prozess und organisatorischer Aufgabensteuerung reduziert Schnittstellenverluste zwischen Entwicklung und Koordination.

Die Evaluation belegt zugleich, dass technische Qualität nicht ausschließlich über Tooling entsteht. Der größte Effekt ergibt sich dort, wo präzise Regeln, klare Rollen und einheitliche Arbeitsabläufe zusammengeführt werden. Für die [O-SW](#) bedeutet das, dass methodische Disziplin im Prozess genauso relevant ist wie die Auswahl der Werkzeuge.

11.2 Limitationen und Validität

Die Aussagekraft der Ergebnisse unterliegt mehreren Limitationen:

- **Interne Validität:** Die Untersuchung erfolgt in einem konkreten Unternehmenskontext mit begrenzter Teamgröße; individuelle Arbeitsstile können Ergebnisse mitprägen.
- **Externe Validität:** Die Übertragbarkeit auf andere Domänen ist nicht vollständig gegeben, insbesondere bei abweichenden Toolketten oder Release-Modellen.
- **Konstruktvalidität:** Ein Teil der Bewertung basiert auf semiquantitativen Trendkategorien statt auf durchgängigen numerischen Zeitreihen.
- **Plattformeinfluss:** Beobachtete Effekte im Review-Prozess sind teilweise durch GitHub-Diff-Mechanismen bestimmt und nicht allein durch Copilot-Instruktionen steuerbar.

Zur Minderung dieser Effekte wurden Iterationen klar getrennt, Maßnahmen isoliert bewertet und Befunde mit realen Repository-Änderungen abgeglichen. Dennoch bleiben Restrisiken, die in zukünftigen Auswertungszyklen mit erweiterten Datensätzen weiter reduziert werden sollten.

11.3 Wirtschaftliche Einordnung

Die Einführung verursacht zunächst Mehraufwand durch Konzeption, Schulung und disziplinierte Prozessanwendung. Dem gegenüber stehen wiederkehrende Nutzenpotenziale im Betrieb:

- frühere Erkennung formaler Qualitätsmängel,
- geringerer Abstimmungsaufwand bei Priorisierung und Freigabe,
- bessere Nachvollziehbarkeit für Übergaben, Onboarding und Fehlersuche.

Für die O-SW wird die Wirtschaftlichkeit als pragmatische Kosten-Nutzen-Prognose auf Basis zusätzlicher Toolkosten und erwarteter Zeiteinsparung modelliert. Die Zeiteinsparung ist dabei eine explizite Schätzung und kann je nach Projekt und Mitarbeiter deutlich variieren.

Zusätzliche Lizenzkosten (monatlich):

- Copilot Business: 25 Lizenzen für Programmierer ($19,99 \text{ EUR} \cdot 25 = 499,75 \text{ EUR}$)
- ZenHub: 7 Lizenzen für Experten und betroffene Nutzer ($4,99 \text{ EUR} \cdot 7 = 34,93 \text{ EUR}$)
- Summe Zusatzkosten/Monat: 534,68 EUR

Nutzenannahme für die O-SW (monatlich):

Zur Berechnung des Nutzens wurden Annahmen getroffen, da keine vollständigen Zeitreihen für alle Projekte vorliegen.

Die Nutzenannahme wird in zwei Bereiche aufgeteilt: erstens beschleunigte Entwicklung und zweitens erleichterte Servicearbeit durch besser lesbaren Code. Vom ersten Bereich sind 25 Programmierer betroffen. Nach einer groben Schätzung von Alexandru und David beträgt die monatliche Einsparung 4 Stunden je Programmierer.

Der zweite Bereich betrifft die Servicearbeit mit etwa 8 Hotline-Mitarbeitern. Eine Schätzung der Abteilungsleitung (Simon Schweighart) geht von rund 3 Stunden Einsparung pro Person und Monat aus, wenn der Code klar lesbar und nachvollziehbar ist. Für beide Bereiche wird ein interner Stundensatz von 85 EUR/h angesetzt.

- Anzahl betroffener Programmierer: 25
- Zeiteinsparung je Programmierer und Monat: 4 h
- Anzahl betroffener Hotliner: 8
- Zeiteinsparung je Hotliner und Monat: 3 h
- Interner Stundensatz: 85 EUR/h

Damit ergibt sich als einfache Nutzenformel:

$$\text{Monatlicher Nutzen} = (25 * 4 + 8 * 3) * 85 = 10.540 \text{ EUR} \quad (1)$$

Abbildung 9 – Nutzenformel (monatlich)

Die Wirtschaftlichkeit ist gegeben, wenn gilt:

$$\text{Monatlicher Nutzen} > \text{Zusatzkosten/Monat} \quad (2)$$

Abbildung 10 – Break-even-Bedingung

Für die hier angesetzten Werte gilt:

$$10.540 \text{ EUR} > 534,68 \text{ EUR} \quad (3)$$

Abbildung 11 – Vergleich Nutzen zu Zusatzkosten

Damit liegt der monatliche Nutzwert rechnerisch deutlich über den zusätzlichen Lizenzkosten. Diese Aussage ist als Prognose zu verstehen und sollte nach Einführung mit Ist-Daten (z. B. PR-Durchlaufzeiten, Nacharbeitszeiten und Serviceaufwände) validiert werden.

Zur Einordnung der Robustheit wurden ergänzend zwei einfache Szenarien betrachtet:

Szenario	Monatlicher Nutzen	Zusatzkosten/ Monat	Saldo/Monat
Konservativ (50 % des geschätzten Nutzens)	5.270 EUR	534,68 EUR	4.735,32 EUR
Realistisch (volle Schätzung)	10.540 EUR	534,68 EUR	10.005,32 EUR

Tabelle 8 – Szenariovergleich zur Wirtschaftlichkeit

Auch im konservativen Szenario bleibt der berechnete Nutzen deutlich oberhalb der zusätzlichen Lizenzkosten.

In der Gesamtabwägung ist der Ansatz wirtschaftlich plausibel, wenn die anfänglichen Einführungsaufwände durch kontinuierliche Nutzung verstetigt werden. Besonders relevant ist dabei die Skalierungswirkung: Mit wachsender Projektanzahl sinken die relativen Zusatzkosten pro Änderung, während standardisierte Abläufe mehrfach nutzbaren Nutzen erzeugen.

11.4 Governance-Empfehlungen für den Dauerbetrieb

Für einen stabilen Langzeitbetrieb werden folgende Leitplanken empfohlen:

- verbindliche Merge-Kriterien je Zielbranch,
- regelmäßige Pflege und Versionierung der Conventions,
- klar definierte Rollen für Priorisierung, Review und Freigabe,
- periodische Prozess-Reviews anhand der in Tabelle 2 beschriebenen Indikatoren.

Diese Governance-Elemente sichern die Dauerhaftigkeit der eingeführten Methodik und reduzieren das Risiko einer schleichenden Rückkehr zu uneinheitlichen Arbeitsweisen.

12

Fazit

Erkenntnis: Im Rahmen dieser Arbeit wurden für die identifizierten Problemfelder Branching, Management und Code-Review konkrete Maßnahmen konzipiert und in den Projektkontext überführt. Die Einführung eines angepassten Git-Flow-Modells, der Einsatz von Copilot-gestützten Reviews sowie die Einbindung von ZenHub adressieren sowohl technische als auch organisatorische Schwachstellen der bisherigen Arbeitsweise.

Die Einordnung der erwarteten Prozesswirkung kann ergänzend auf etablierten Befunden zu Delivery- und Team-Performance gestützt werden [8].

Implikation: Die durchgeführten Tests und Abstimmungen zeigen, dass die gewählten Ansätze grundsätzlich geeignet sind, Transparenz, Nachvollziehbarkeit und Qualität in der Zusammenarbeit zu erhöhen. Gleichzeitig wurde deutlich, dass die Wirksamkeit der Maßnahmen von klar formulierten Richtlinien, konsistenter Anwendung im Team und einer belastbaren Schulungs- und Dokumentationsbasis abhängt.

Im Ergebnis lässt sich festhalten, dass vor allem das Zusammenspiel der Maßnahmen den größten Nutzen erzeugt. Eine Branching-Strategie allein verbessert die technische Ordnung, entfaltet ihre volle Wirkung aber erst in Verbindung mit nachvollziehbaren Reviews und strukturierter Aufgabensteuerung. Umgekehrt bleibt Projektmanagement ohne klare Entwicklungs- und Freigaberegeln in kritischen Phasen zu wenig belastbar. Die in dieser Arbeit entwickelte Kombination schafft daher eine tragfähige Grundlage für einen konsistenten End-to-End-Prozess von der Aufgabenplanung bis zur Integration in stabile Zielbranches.

Gleichzeitig wurden Grenzen sichtbar. KI-gestützte Reviews erhöhen die Prüfbreite deutlich, ersetzen aber keine fachliche Bewertung durch erfahrene Entwickler, insbesondere bei domänenspezifischen SPS-Logiken und bei semantisch komplexen Änderungen. Darüber hinaus erzeugt die stärkere Standardisierung anfangs zusätzlichen Abstimmungsaufwand, bis Rollen, Namensregeln und Freigabekriterien im Team verlässlich eingeübt sind. Diese Einführungsphase ist kein Nachteil des Ansatzes, sondern eine notwendige Voraussetzung für die spätere Prozessstabilität.

Als nächster Schritt empfiehlt sich die fortlaufende Messung ausgewählter Prozessindikatoren, beispielsweise Durchlaufzeiten von Pull Requests, Anteil ungeplanter Nacharbeiten nach Merge sowie Häufigkeit von Konflikten in integrationsnahen Branches. Auf dieser Basis kann die eingeführte Methodik datenbasiert weiterentwickelt werden. Parallel sollte die Wissensbasis im Wiki kontinuierlich gepflegt werden, damit neue Teammitglieder den Standard schnell übernehmen und bestehende Mitarbeitende bei Abweichungen auf konsolidierte Referenzen zurückgreifen können.

Ausblick/Übergang: Insgesamt legt die Arbeit damit eine praktikable Grundlage für einen einheitlichen, GitHub-basierten Entwicklungs- und Steuerungsprozess in der [fpt Systems GmbH](#).

A Literatur

- [1] G. Gousios, M. Pinzger, und A. van Deursen, „An Exploratory Study of the Pull-based Software Development Model“, in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, 2014, S. 345–355. doi: [10.1145/2568225.2568260](https://doi.org/10.1145/2568225.2568260).
- [2] V. Driessen, „A successful Git branching model“. [Online]. Verfügbar unter: <https://nvie.com/posts/a-successful-git-branching-model/>
- [3] A. Bacchelli und C. Bird, „Expectations, Outcomes, and Challenges of Modern Code Review“, in *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, 2013, S. 712–721. doi: [10.1109/ICSE.2013.6606617](https://doi.org/10.1109/ICSE.2013.6606617).
- [4] S. McIntosh, Y. Kamei, B. Adams, und A. E. Hassan, „The Impact of Code Review Coverage and Code Review Participation on Software Quality“, *Empirical Software Engineering*, Bd. 21, Nr. 5, S. 1863–1905, 2016, doi: [10.1007/s10664-015-9388-4](https://doi.org/10.1007/s10664-015-9388-4).
- [5] GitHub, Inc., „About pull request reviews“. [Online]. Verfügbar unter: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/code-quality-features/about-pull-request-reviews>
- [6] GitHub, Inc., „About protected branches“. [Online]. Verfügbar unter: <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches>
- [7] ZenHub Technologies Ltd., „ZenHub Help Center“. [Online]. Verfügbar unter: <https://help.zenhub.com/>
- [8] N. Forsgren, J. Humble, und G. Kim, *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.

B Glossar

E-SW: Entwicklung Software - Abteilung bei der fpt Systems GmbH, welche die Entwicklung neuer FlexOp-Versionen und anderer Hochsprachenprojekte übernimmt [9](#), [11](#), [11](#), [12](#), [12](#), [19](#), [20](#), [20](#)

FlexOp: Firmeneigene Nutzeroberfläche zu Steuerung und Verwaltung von Anlagen [10](#), [12](#)

O-SW: Operations Software - Abteilung bei der fpt Systems GmbH, welche die Programmierung von SPS Projekten übernimmt [4](#), [6](#), [7](#), [9](#), [10](#), [11](#), [11](#), [11](#), [11](#), [12](#), [12](#), [12](#), [16](#), [18](#), [19](#), [19](#), [48](#), [52](#)

TwinCat: Aktuelle Version der Entwicklungsumgebung für Beckhoff SPS [10](#)

fpt Systems GmbH: Tochterfirma der FPT Robotik GmbH & Co. KG, welche für die Softwareentwicklung verantwortlich ist. [11](#), [57](#)

Sperrvermerk

Die vorliegende Arbeit mit dem Titel

**Konzeption und Implementierung eines GitHub-basierten Branching-,
Review- und Projektmanagementprozesses mit KI-gestützter Code-Compliance**

enthält unternehmensinterne bzw. vertrauliche Informationen der fpt Systems GmbH, ist deshalb mit einem Sperrvermerk versehen und wird ausschließlich zu Prüfungszwecken im Studiengang Elektrotechnik der Duale Hochschule Baden-Württemberg Ravensburg Campus Friedrichshafen vorgelegt.

Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungsprozesses und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anders lautende Genehmigung der Ausbildungsstätte (fpt Systems GmbH) vorliegt.

Amtzell 24.08.2026



Florian Germann

Selbstständigkeitserklärung

Gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden- Württemberg vom 29.09.2017. Ich versichere hiermit, dass ich meine Arbeit mit dem Thema:

Konzeption und Implementierung eines GitHub-basierten Branching-, Review- und Projektmanagementprozesses mit KI-gestützter Code-Compliance

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass alle eingereichten Fassungen übereinstimmen.

Amtzell 24.08.2026



Florian Germann